

MEDICAL DEVICE MARKETING

Importance

Medical device marketing connects **technology with healthcare needs**. A technically excellent device may fail commercially if it is too expensive, difficult to use, unavailable through healthcare channels, or does not provide sufficient clinical value.

2. Customers

A **customer** is a person, organization, or institution that purchases, uses, influences the purchase of, or benefits from a product or service.

In healthcare, the customer is not always the same person as the end user. For example, a patient may use a medical device, while the hospital purchases it and a doctor recommends it.

Types of customers in healthcare

1. Patient

The patient is the ultimate beneficiary of most healthcare products and services. Their comfort, safety, affordability, and treatment outcomes are important.

2. Doctor/Physician

Doctors may recommend, prescribe, evaluate, or influence the selection of medical devices.

3. Hospital

Hospitals are major institutional customers. They may purchase equipment such as patient monitors, ultrasound systems, ventilators, and diagnostic devices.

4. Biomedical Engineer

Biomedical engineers evaluate the technical performance, maintenance requirements, compatibility, and safety of medical equipment.

5. Procurement Department

The procurement team handles purchasing, quotations, vendor comparison, negotiations, and purchase orders.

6. Nurses and Technicians

They are important users of many medical devices and can influence purchasing decisions based on usability and reliability.

MEDICAL DEVICE MARKETING

7. Distributors and Dealers

They purchase products from manufacturers and distribute them to hospitals, clinics, pharmacies, and other customers.

Customer needs

Customer needs may include:

- Safety
- Clinical effectiveness
- Reliability
- Ease of use
- Affordability
- Durability
- Technical support
- Training
- After-sales service
- Availability of spare parts
- Regulatory compliance

Example

For a **portable ECG device**:

- Patient → receives the benefit
- Doctor → interprets the ECG
- Nurse → may operate the device
- Hospital → purchases the device
- Biomedical engineer → evaluates and maintains it
- Distributor → supplies the device

MEDICAL DEVICE MARKETING

Thus, understanding the **different roles of customers** is essential for successful medical device marketing.

Decision Making

Introduction

Decision making is the process of identifying a problem or opportunity, collecting relevant information, evaluating alternatives, selecting the best option, implementing the decision, and evaluating its outcome.

In healthcare and medical device marketing, decision making is complex because decisions may involve **patient safety, clinical effectiveness, cost, regulatory compliance, technology, and ethical considerations**.

Steps in the Decision-Making Process

1. Identify the problem

The first step is to clearly understand what problem needs to be solved.

Example:

A hospital experiences delays in detecting abnormal oxygen levels in patients.

2. Define the objective

The organization should determine what it wants to achieve.

For example:

- Improve patient monitoring
- Reduce response time
- Improve accuracy
- Reduce operating costs

3. Collect information

Relevant information is collected from:

- Doctors
- Nurses

MEDICAL DEVICE MARKETING

- Scalability
- Customer support
- **6. Evaluate alternatives**
- Each alternative is compared against the established criteria.
- For example:

Criteria	Device A	Device B	Device C
Cost	High	Medium	Low
Accuracy	High	High	Medium
Ease of use	Medium	High	High
Maintenance	High	Medium	Low
Connectivity	Medium	High	High

7. Select the best alternative

The alternative providing the best balance between **clinical value, technical performance, cost, safety, and customer requirements** is selected.

8. Implement the decision

The selected option is put into practice.

Implementation may include:

- Procurement
- Installation
- Training
- Testing
- Validation
- Documentation

9. Monitor the result

After implementation, performance should be monitored.

MEDICAL DEVICE MARKETING

Parameters may include:

- Device accuracy
- User satisfaction
- Patient outcomes
- Downtime
- Maintenance cost
- Return rate

10. Review and improve

If the decision does not produce the expected result, corrective action should be taken.

Factors Affecting Decision Making

Technical factors

- Accuracy
- Reliability
- Compatibility
- Performance
- Technology maturity

Economic factors

- Purchase cost
- Operating cost
- Maintenance cost
- Return on investment

Clinical factors

- Patient safety
- Clinical effectiveness

MEDICAL DEVICE MARKETING

- Ease of use
- Patient outcomes

Regulatory factors

- Regulatory approval
- Standards
- Quality requirements
- Documentation

Ethical factors

- Patient rights
- Privacy
- Informed consent
- Transparency

Organizational factors

- Budget
- Infrastructure
- Staff availability
- Management policy

Example

Suppose a hospital wants to purchase a new ultrasound machine. The decision-making process would involve:

Problem → Need identification → Market research → Identification of alternatives → Technical evaluation → Clinical evaluation → Cost comparison → Regulatory verification → Vendor evaluation → Purchase decision → Installation → Training → Performance evaluation.

MEDICAL DEVICE MARKETING

10. Evaluate the product after purchase

Follow-Up

Follow-up is the process of maintaining communication with a customer after initial contact, product demonstration, quotation, or purchase.

Importance of follow-up

- Builds customer relationships
- Identifies customer problems
- Provides technical support
- Improves customer satisfaction
- Encourages repeat purchases
- Helps identify new requirements
- Provides feedback for product improvement

Follow-up activities

Before purchase:

- Call the customer
- Provide product information
- Answer technical questions
- Provide quotation
- Arrange demonstrations

After purchase:

- Installation
- User training
- Performance verification
- Maintenance support
- Collect feedback

MEDICAL DEVICE MARKETING

- Handle complaints
- Provide warranty service

Example

After selling a patient monitor to a hospital, the company should contact the biomedical department to confirm installation, train users, check performance, and collect feedback.

Fundamentals of Reimbursement

Introduction

Healthcare reimbursement refers to the **payment made to healthcare providers, hospitals, clinics, or other healthcare organizations for providing healthcare services or products.**

It is an important component of healthcare economics because patients and healthcare providers may not directly bear the entire cost of treatment.

Major Participants

1. **Patient** – Receives healthcare.
2. **Healthcare provider** – Provides treatment or service.
3. **Hospital/Clinic** – Delivers healthcare services.
4. **Insurance company/payer** – May pay some or all eligible costs.
5. **Government** – Provides or finances healthcare through public schemes.
6. **Medical device company** – Supplies products used in treatment.

Basic Reimbursement Process

Patient receives treatment → Provider documents service → Claim is submitted → Payer evaluates claim → Claim is approved/rejected → Payment is made.

Types of Reimbursement

1. Fee-for-Service

Payment is made for individual healthcare services provided.

MEDICAL DEVICE MARKETING

2. Capitation

A fixed payment is provided for each patient for a defined period.

3. Bundled Payment

A single payment covers multiple services associated with a treatment or procedure.

4. Diagnosis-Related Payment

Payment is associated with the diagnosis or treatment category rather than each individual service.

5. Government-funded reimbursement

Government healthcare programs may reimburse hospitals or providers for eligible treatments.

Importance

Reimbursement affects:

- Product affordability
- Hospital purchasing decisions
- Patient access
- Medical device adoption
- Healthcare provider revenue
- Market potential

Example

A technologically advanced medical device may have excellent clinical performance but limited market adoption if patients or hospitals cannot afford it or if the relevant healthcare payment system does not cover its use.

MEDICAL DEVICE MARKETING

Healthcare Economics

Healthcare economics is the study of how **limited healthcare resources are allocated to satisfy the healthcare needs of individuals and society.**

Resources include:

- Money
- Medical equipment
- Healthcare professionals
- Medicines
- Hospital beds
- Infrastructure
- Time

Importance

Healthcare economics helps answer questions such as:

- Is the technology affordable?
- Does it provide sufficient clinical benefit?
- What is the cost of treatment?
- Is investment in the technology justified?
- How can limited healthcare resources be used efficiently?

Major concepts

Cost: Money or resources required for a healthcare product or service.

Benefit: Positive outcome obtained from healthcare intervention.

Cost-effectiveness: Comparison between cost and health outcome.

Cost-benefit: Comparison of monetary cost with monetary benefit.

Cost-utility: Comparison of cost with outcomes such as quality-adjusted life years.

Opportunity cost: The benefit lost when one option is selected instead of another.

MEDICAL DEVICE MARKETING

Example

A hospital may compare two patient-monitoring systems. One system may be cheaper initially, while another may have higher purchase cost but lower maintenance and better clinical performance. Healthcare economics helps determine which option provides greater overall value.

The Letter of Law, Ethics and Responsibilities

Introduction

Medical devices directly or indirectly affect human health and safety. Therefore, companies and healthcare professionals must comply with **laws, regulations, ethical principles, and professional responsibilities**.

The **letter of the law** refers to strict compliance with the actual legal requirements applicable to a product, organization, or activity. However, ethical responsibility goes beyond merely following minimum legal requirements.

A. Letter of Law

The letter of law means following established legal requirements exactly as specified by the relevant authorities.

For medical devices, legal requirements may cover:

- Product safety
- Quality management
- Manufacturing
- Clinical evaluation
- Regulatory approval
- Labelling
- Advertising
- Data protection
- Post-market surveillance

MEDICAL DEVICE MARKETING

- Reporting of adverse events

Why legal compliance is important

1. Protects patients
2. Ensures product safety
3. Builds customer confidence
4. Prevents legal penalties
5. Reduces product liability
6. Supports market acceptance
7. Protects the reputation of the organization

Example

A company should not market a medical device for a clinical indication that has not been legally authorized merely because the device appears technically capable of performing that function.

B. Ethics

Ethics refers to principles that guide people and organizations to distinguish between **right and wrong conduct**.

Important ethical principles include:

1. Beneficence

The product should provide benefit to the patient.

2. Non-maleficence

The product should not unnecessarily cause harm.

3. Autonomy

Patients should have the right to make informed decisions about their healthcare.

4. Justice

Healthcare resources and services should be provided fairly.

MEDICAL DEVICE MARKETING

5. Honesty

Manufacturers and healthcare professionals should provide accurate information.

6. Transparency

Limitations, risks, costs, and performance should not be deliberately hidden.

C. Responsibilities of Medical Device Companies

Product responsibility

The manufacturer should ensure:

- Safe design
- Reliable manufacturing
- Quality control
- Proper testing
- Appropriate documentation

Marketing responsibility

Companies should not:

- Make false claims
- Hide product limitations
- Mislead healthcare professionals
- Exaggerate clinical benefits
- Provide unsupported performance claims

Patient responsibility

Patient safety should remain a primary consideration.

Environmental responsibility

Companies should consider:

- Safe disposal

MEDICAL DEVICE MARKETING

- Electronic waste
- Sustainable materials
- Energy consumption
- Environmental impact

D. Responsibilities of Healthcare Professionals

Healthcare professionals should:

- Prioritize patient welfare
- Maintain confidentiality
- Use devices appropriately
- Provide accurate information
- Report adverse events
- Maintain professional competence
- Avoid conflicts of interest

E. Conflict of Interest

A conflict of interest occurs when personal, financial, or organizational interests may influence professional judgment.

Example:

A healthcare professional recommending a particular device because they receive an undisclosed financial benefit from the manufacturer represents an ethical concern.

F. Ethical Marketing

Ethical marketing requires:

1. Truthful product information
2. Evidence-based claims
3. Transparent pricing
4. No misleading advertisements

MEDICAL DEVICE MARKETING

5. Respect for patient privacy
6. Responsible promotion
7. Proper disclosure of risks
8. No inappropriate influence on purchasing decisions

Law vs Ethics

Law	Ethics
Legally mandatory	Based on moral/professional principles
Defined by legal authorities	Influenced by professional and societal values
Violation may result in legal penalties	Violation may damage trust and reputation
Represents minimum legal requirements	May demand a higher standard of conduct

Business Plan

Introduction

A business plan is a structured document that explains **what a business intends to achieve, how it will achieve it, who its customers are, what resources are required, and whether the business is financially viable.**

For a medical device company, a business plan connects the **healthcare problem → product → market → customer → financial model → implementation strategy.**

A typical business plan includes:

1. Executive summary
2. Problem identification
3. Product/service description
4. Customer identification
5. Market research
6. Competitor analysis

MEDICAL DEVICE MARKETING

7. Opportunity assessment
8. Marketing strategy
9. Financial analysis
10. Project scope
11. Risk analysis
12. Implementation plan
13. Stage-gate process

Market Research

Market research is the systematic process of collecting and analyzing information about **customers, competitors, market demand, pricing, trends, and business opportunities.**

Objectives

- Understand customer needs
- Estimate market size
- Identify competitors
- Determine customer preferences
- Understand pricing
- Identify market gaps
- Reduce business risk

Types of Market Research

Primary research

Information collected directly from customers.

Examples:

- Interviews
- Questionnaires
- Surveys

MEDICAL DEVICE MARKETING

- Focus groups
- Product demonstrations
- Hospital visits

Secondary research

Information obtained from existing sources.

Examples:

- Research papers
- Government reports
- Industry reports
- Published statistics
- Company websites
- Market databases

Market Research Process

Identify problem → Define research objective → Collect information → Analyze data →

Identify market opportunity → Make business decision

Example

For an **IoT-based sepsis monitoring device**, market research may determine:

- Number of hospitals requiring continuous monitoring
- Existing monitoring systems
- Doctors' requirements
- Device cost expectations
- Competitor products
- Required regulatory approvals
- Potential customers

MEDICAL DEVICE MARKETING

Accessing Opportunity / Assessing Opportunity

Assessing an opportunity means evaluating whether a particular idea or market need has sufficient **technical, clinical, commercial, and financial potential**.

Major factors

1. Problem

Is there a genuine healthcare problem?

2. Market size

How many potential customers exist?

3. Customer need

Are customers willing to use or purchase the solution?

4. Competition

Are similar products already available?

5. Competitive advantage

What makes the proposed product better?

6. Technical feasibility

Can the product actually be developed?

7. Regulatory feasibility

Can the product obtain necessary approvals?

8. Financial feasibility

Can the company develop and sell the product profitably?

9. Scalability

Can production be increased when demand grows?

10. Risk

What technical, financial, clinical, regulatory, and market risks exist?

Opportunity Assessment Example

MEDICAL DEVICE MARKETING

For a smart prosthetic socket:

Problem: Conventional sockets may cause discomfort.

Opportunity: Develop a sensor-based socket that monitors pressure and fit.

Customers: Amputees, rehabilitation centers, hospitals.

Technology: Pressure sensors + IoT + data analysis.

Market advantage: Real-time monitoring and personalized fitting.

Feasibility: Requires technical validation, clinical testing, manufacturing capability, and regulatory compliance.

Financial Analysis

Financial analysis evaluates whether a proposed business or project is **economically viable and financially sustainable**.

Major components

1. Initial investment

Money required to start the project.

Examples:

- Equipment
- Software
- Manufacturing setup
- Research and development
- Regulatory expenses
- Infrastructure

2. Operating costs

Costs required for regular operation.

Examples:

MEDICAL DEVICE MARKETING

- Salaries
- Electricity
- Maintenance
- Raw materials
- Marketing
- Transportation

3. Revenue

Income generated through product sales or services.

$$\text{Revenue} = \text{Selling Price} \times \text{Number of Units Sold}$$

4. Profit

$$\text{Profit} = \text{Revenue} - \text{Total Cost}$$

5. Break-even analysis

Break-even point is the point where total revenue equals total cost.

$$\text{Break-even units} = \text{Fixed Cost} \div (\text{Selling Price} - \text{Variable Cost per Unit})$$

6. Return on Investment

ROI measures the return obtained from an investment.

$$\text{ROI} = (\text{Net Return} \div \text{Investment}) \times 100$$

7. Cash flow

Cash flow represents the movement of money into and out of the business.

8. Financial risk

Possible risks include:

- Low sales
- Increased manufacturing cost
- Delayed regulatory approval

MEDICAL DEVICE MARKETING

- High R&D expenditure
- Competition
- Supply-chain problems

Example

If a medical device costs ₹10,000 to manufacture and is sold for ₹18,000, the difference contributes toward covering fixed costs and generating profit.

Project Scope

Project scope defines **what a project will and will not cover**.

It establishes the project's boundaries and prevents uncontrolled expansion of activities.

Components of project scope

1. Project objective
2. Deliverables
3. Requirements
4. Activities
5. Timeline
6. Resources
7. Budget
8. Responsibilities
9. Constraints
10. Exclusions
11. Acceptance criteria

Example

For an IoT patient monitoring project:

Objective: Develop a system for real-time monitoring of vital parameters.

MEDICAL DEVICE MARKETING

Inputs: Heart rate, SpO₂, temperature, etc.

Technology: Sensors + microcontroller + communication + cloud platform.

Output: Real-time display and alerts.

Deliverable: Functional prototype and validated monitoring system.

Exclusion: Full commercial regulatory approval may fall outside the initial prototype-development phase.

Importance

A clearly defined project scope:

- Controls cost
- Controls time
- Defines responsibilities
- Prevents scope creep
- Helps measure progress
- Improves project management

Stage Gate Process

Introduction

The **Stage-Gate process** is a structured product development methodology in which a project progresses through a series of **stages separated by decision gates**.

Each stage performs specific activities, while each gate evaluates whether the project should:

- Continue
- Stop
- Be modified
- Be delayed
- Be redirected

MEDICAL DEVICE MARKETING

It is particularly useful in medical device development because healthcare products involve significant **technical, clinical, regulatory, financial, and market risks**.

General Stage-Gate Model

Idea → Gate 1 → Concept → Gate 2 → Development → Gate 3 → Testing/Validation → Gate 4 → Launch → Gate 5 → Post-launch review

Stage 1 – Idea Generation

Potential product ideas are identified.

Sources include:

- Customer complaints
- Clinical problems
- Doctor feedback
- Research
- New technologies
- Market gaps

Gate 1: Determine whether the idea deserves further investigation.

Stage 2 – Concept Development

The selected idea is converted into a preliminary product concept.

Activities include:

- Customer identification
- Market research
- Competitor analysis
- Technical feasibility
- Initial risk assessment
- Preliminary business case

Gate 2: Decide whether the concept is sufficiently attractive and feasible.

MEDICAL DEVICE MARKETING

Stage 3 – Product Development

The actual product is designed and developed.

Activities include:

- Engineering design
- Prototype development
- Software development
- Component selection
- Risk management
- Design documentation

Gate 3: Evaluate whether the product is ready for testing and validation.

Stage 4 – Testing and Validation

The developed product is tested to determine whether it meets requirements.

Testing may include:

- Functional testing
- Safety testing
- Performance testing
- Usability testing
- Verification
- Validation
- Clinical evaluation where applicable

Gate 4: Decide whether the product is ready for commercialization.

Stage 5 – Product Launch

The product is introduced into the market.

Activities include:

MEDICAL DEVICE MARKETING

- Regulatory compliance
- Manufacturing
- Packaging
- Marketing
- Sales
- Distribution
- Customer training

Gate 5: Evaluate commercial performance.

Stage 6 – Post-Launch Review

After commercialization, the company evaluates:

- Sales
- Customer satisfaction
- Product complaints
- Clinical performance
- Maintenance
- Profitability
- Adverse events
- Opportunities for improvement

Decision Criteria at Gates

- Each gate may consider:

Factor	Question
Market	Is there sufficient demand?
Customer	Does it solve a real customer problem?
Technical	Can it be developed successfully?
Clinical	Does it provide meaningful healthcare benefit?

MEDICAL DEVICE MARKETING

Regulatory	Can it satisfy applicable requirements?
Financial	Is it economically viable?
Risk	Are risks acceptable?
Competition	Does it have sufficient competitive advantage?

Advantages of Stage-Gate Process

1. Reduces risk

Problems can be identified before large investments are made.

2. Controls expenditure

Projects that are unlikely to succeed can be stopped early.

3. Improves product quality

Each stage has defined requirements and evaluation criteria.

4. Supports informed decisions

Management receives structured information before approving the next stage.

5. Improves resource allocation

Money, people, equipment, and time can be directed toward promising projects.

6. Supports regulatory planning

Medical device regulatory and quality requirements can be considered throughout development.

7. Improves commercialization

Market and customer requirements are considered before product launch.

Example – Smart Patient Monitoring Device

Idea: Develop a wearable device for continuous monitoring.



Concept: Identify parameters such as heart rate, SpO₂, temperature, and activity.



Development: Develop sensors, embedded system, communication, and software.



Testing: Verify accuracy, safety, usability, and connectivity.



Regulatory/Commercial Gate: Evaluate compliance and business viability.



Launch: Manufacture and sell the product.



MEDICAL DEVICE MARKETING

Post-launch: Collect customer feedback and monitor product performance.



KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)

Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A" Grade



JAVA PROGRAMMING

BY

DR. R. RAMALAKSHMI

DR. K. MURUGESWARI

MR. M. K. NAGARAJAN

MRS. A. M. GURUSIGAAMANI

Course Outline



CO 1. Understand the object oriented concepts using java



CO2. Apply fundamental programming concepts of Java to develop stand alone applications



CO 3. Solve Real World Problem through reusable and error free code.



CO 4. Design and develop distributed applications.



CO 5. Implement window based applications using event handling mechanisms

Course description

Java still has its importance and popularity in the software industry, in spite of recent development of several high level languages. It has an excellent support of high-level as well as low-level functionality, which makes it suitable for many applications. This course provides students with a comprehensive study of the Java programming language. Instead of Classroom lectures, practical classes stress the strengths of Java, which provide the students with the means of writing efficient, maintainable, and portable code. A course with mini project is one of the great learning opportunities to develop students in various aspects. This course will also help the students during their placement sittings, as most of the companies test proficiency in programming using Java.

UNIT I OOP IN JAVA & INHERITANCE

Object Oriented Programming Concepts - OOP in Java – Characteristics of Java – Fundamental Programming Structures in Java – Defining classes in Java – Comments, Data Types, Variables, Operators, Control Flow, Arrays - constructors, methods -access specifiers - static members – Packages- Inheritance – Super classes- sub classes –Protected members – constructors in sub classes- Strings.

UNIT II EXCEPTION HANDLING AND I/O

Exceptions - exception hierarchy - throwing and catching exceptions – built-in exceptions, creating own exceptions, Stack Trace Elements-Input / Output Basics – Streams – Byte streams and Character streams – Reading and Writing Console – Reading and Writing Files - abstract classes and methods - final methods and classes

UNIT III INTERFACES AND MULTITHREADING

Interfaces – defining an interface, implementing interface, differences between classes and interfaces - extending interfaces - Object cloning -inner classes- Differences between multi-threading and multitasking, thread life cycle, creating threads, synchronizing threads, Inter-thread communication, daemon threads - Generic Programming.

UNIT IV AWT AND EVENT DRIVEN PROGRAMMING

AWT Event Hierarchy- Components - Graphics programming – Applets-Frame – working with 2D shapes - Using color, fonts, and images - Basics of event handling - event handlers - adapter classes - actions - mouse events - Introduction to Swing – layout management - Swing Components – Windows – Menus – Dialog Boxes.

UNIT V NETWORKING AND JDBC

Networking Basics - The Networking Classes and Interfaces -TCP/IP Client Sockets URL TCP/IP Server Sockets – **Datagrams** - A Relational Database Overview - JDBC Introduction - JDBC Product Components - JDBC Architecture - Case studies.



Unit 1

Outcomes

Define classes.

Create objects using default and parameterized constructors.

Run stand alone applications using programming constructs of Java

Inherit the properties of an existing classes.

Hide the members of a class using access specifiers

Use predefined packages and create own package to group related classes.

Unit 1 Outline



Lesson 1. Object Oriented programming Concepts



Lesson 2. Fundamental programming Constructs in Java



Lesson 3. Access Specifiers



Lesson 4. Constructors



Lesson 5. Inheritance



Lesson 6. Packages

Briefs about OOP concepts, Characteristics of Java , Fundamental Programming Structures in Java , Defining classes in Java, Comments, Data Types, Variables, Operators, Control Flow, Arrays, constructors, methods, access specifiers, static members, Packages, Inheritance, Super classes, sub classes, Protected members, constructors in sub classes, Strings.

Course Progress (use before starting new lesson)

Lesson 1. Object Oriented programming Concepts

Lesson 2. Fundamental programming Constructs

Lesson 3. Access Specifiers

Lesson 4. Constructors

Lesson 5. Inheritance



Introduction

- Object-Oriented Programming is a programming paradigm that uses “Objects” and their interactions to design applications.
- It simplifies the software development and maintenance by providing some concepts.



Object Oriented Programming Concepts

- . Object
- . Class
- . Abstraction
- . Encapsulation
- . Inheritance
- . Polymorphism



Object

- . An object is defined as the Realworld Entity.
- . It consists of properties, tasks performed by objects.
- . It also defined as instance of a Class

Ex: Human

Properties ☐ Name, Colour, Height

Tasks Performed ☐ Walk(), Run(), Work(), Read()

Creating an object

Creating an object of the class is also called as instantiating an object. Here, we create an object called “s” and print the value of regno.

```
public class student1
{
    int regno=5;
    public static void main(String args[])
    {
        student1 s=new student1();
        System.out.println(s.regno);
    }
}
```

OUTPUT OF OBJECT CREATION

- Output

```
C:\Users\admin\Desktop\dhilipjava>javac student1.java
```

```
C:\Users\admin\Desktop\dhilipjava>java student1  
5
```

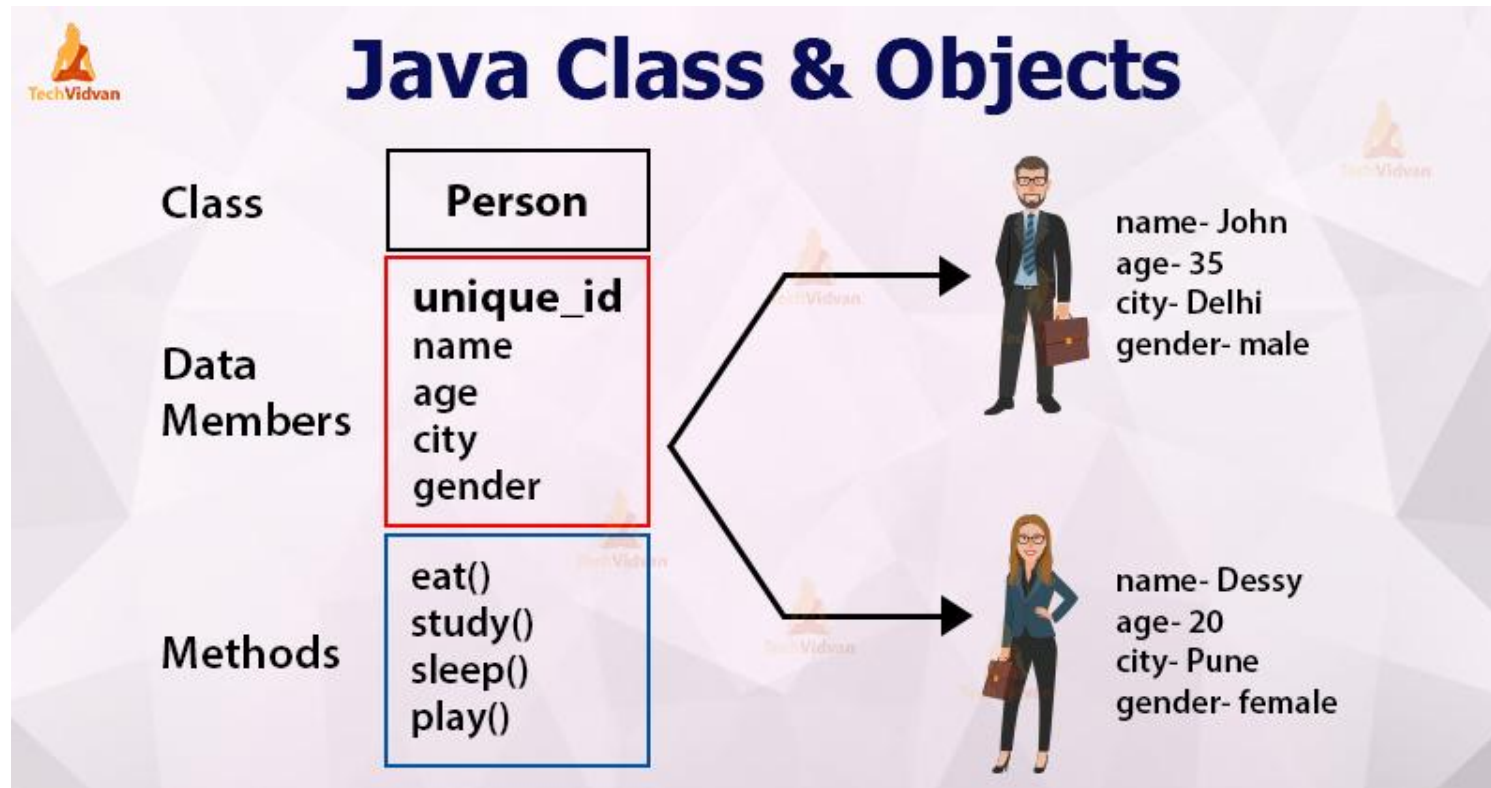
```
C:\Users\admin\Desktop\dhilipjava>_
```



Class

- ❖ Class is a collection of data members and member functions.
- ❖ It is called as the blueprint that defines the variables and methods.
- ❖ When creating instance of a class the system allocates enough memory for the object.

Java Class & Objects



source:techvidvan.com

Syntax of Class:

```
class class_name
{
    type instance_variable;
    type methodname1(parameter_list)
    {
        // body of method
    }
}
```


Example of a Class

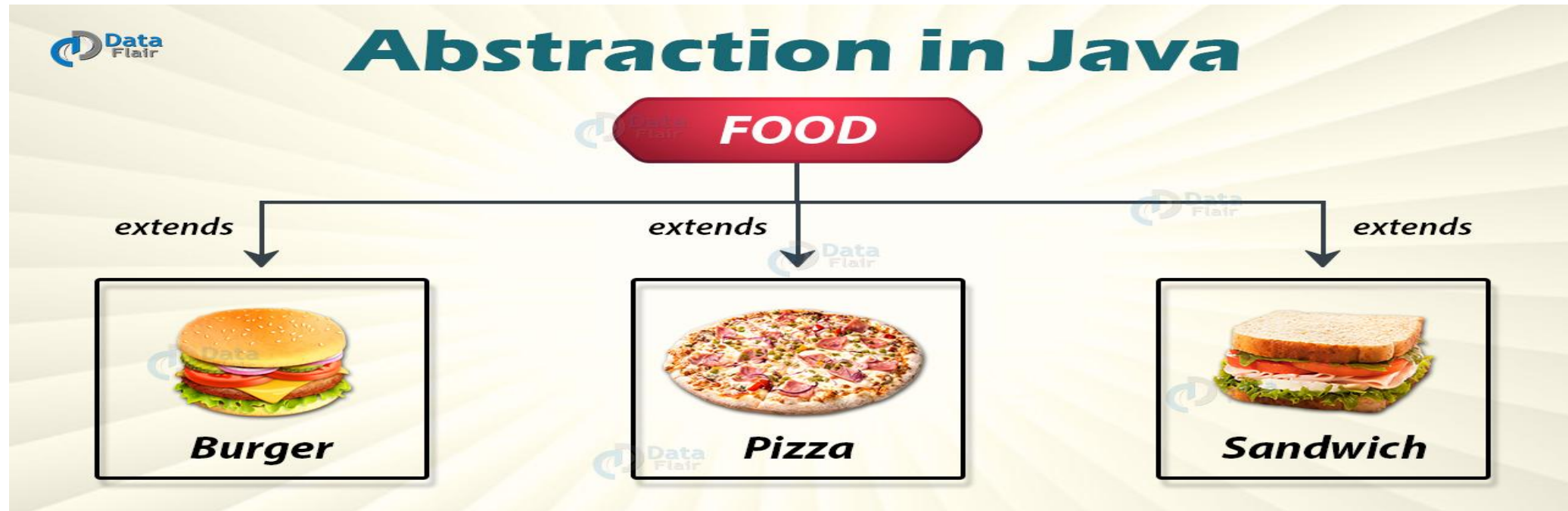
```
Class Student
{
    int regno,marks;
    char name;
    void totalmarks ()
    {
        . . . . .
    }
}
```

Abstraction

It shows only essential things and hides unnecessary information from the user. It helps the user in reducing programming complexity. Two way of abstraction are as follows

- ❖ Data Abstraction
- ❖ Process Abstraction

Abstraction



Source: data-flair.training.com

Example of Abstraction

```
abstract class Bike
{
    abstract void run();
}
class Honda4 extends Bike
{
    void run()
    {
        System.out.println("running safely");
    }
    public static void main(String args[])
    {
        Bike obj = new Honda4();
        obj.run();
    }
}
```

Output of Abstraction

```
C:\Users\admin\Desktop\dhilipjava>javac Honda4.java
```

```
C:\Users\admin\Desktop\dhilipjava>java Honda4  
running safely
```

```
C:\Users\admin\Desktop\dhilipjava>
```



Encapsulation

- ❖ It is a mechanism of wrapping data(variable) and code acting on the data(methods) together as a single unit.
- ❖ It binds together the code and the data it manipulates and keeps both safe from outside.
- ❖ In Java, the **basis of encapsulation is the class.**
- ❖ A **class** defines the structure and behavior (data and code) that will be shared by a set of objects.

Encapsulation Figure

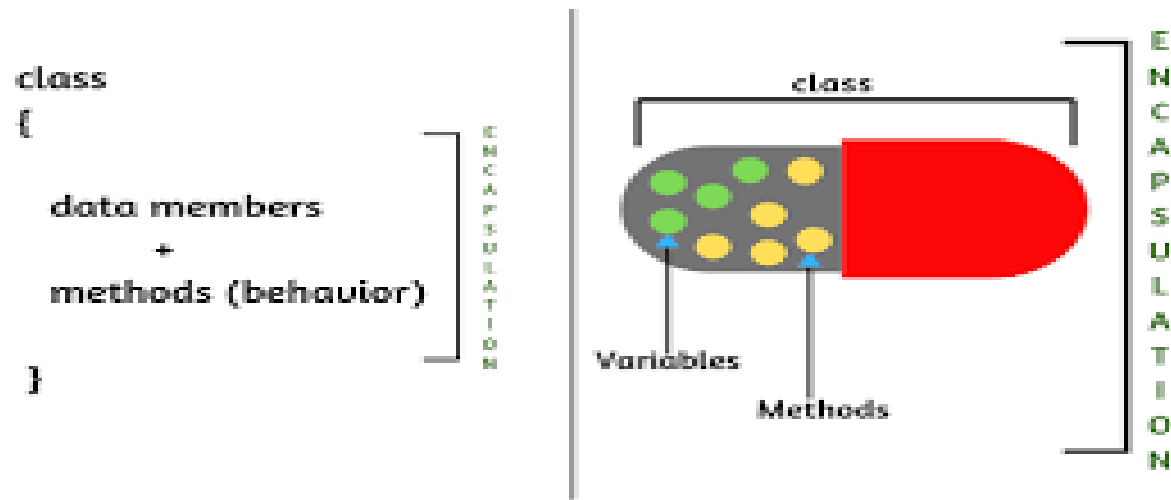


Fig: Encapsulation

Image src:medium.com



Imagesrc:codeeasy.net

Program for Encapsulation

```
public class RunEncap
{
    public static void main(String args[])
    {
        EncapTest encap = new EncapTest();
        encap.setName("James");
        encap.setAge(20);
        encap.setIdNum("12343ms");

        System.out.print("Name : " + encap.getName() + " Age : " + encap.getAge());
    }
}
```


Output of Encapsulation

```
C:\Users\admin\Desktop\dhilipjava>javac EncapTest.java  
C:\Users\admin\Desktop\dhilipjava>javac RunEncap.java  
C:\Users\admin\Desktop\dhilipjava>java RunEncap  
Name : James Age : 20  
C:\Users\admin\Desktop\dhilipjava>
```

Inheritance

- It is the process of deriving a class from the base class.
- For example, one person is getting the behavior of his parent is an example of inheritance.
- If one object acquires the properties of another object.
- This is important because it supports the concept of hierarchical classification.
- Types of inheritance are as follows
 - Single Inheritance
 - Multiple Inheritance
 - Multilevel Inheritance
 - Hierarchal Inheritance
 - Hybrid Inheritance

Program for Inheritance

```
class Doctor
{
    void Doctor_Details()
    {
        System.out.println("Doctor Details...");
    }
}
class Surgeon extends Doctor
{
    void Surgeon_Details()
    {
        System.out.println("Surgen Detail...");
    }
}
```

```
public class Hospital
{
    public static void main(String args[])
    {
        Surgeon s = new Surgeon();
        s.Doctor_Details();
        s.Surgeon_Details();
    }
}
```

Output of Inheritance

```
NAME : DINESH NGC : 20  
C:\Users\admin\Desktop\dhilipjava>javac Hospital.java  
  
C:\Users\admin\Desktop\dhilipjava>java Hospital  
Doctor Details...  
Surgen Detail...  
  
C:\Users\admin\Desktop\dhilipjava>_
```

Polymorphism

- Polymorphism “Poly” means many “morph” means form.
- It is one of the feature in oops that performs single action in different ways. For example class vehicle has method cars().
- In that cars are of different types XUV, ZEDON, HATCHBACK etc
- Two types of polymorphism are there. They are as follows
 - Compile time polymorphism (Static Binding)
 - Run time polymorphism (Dynamic Binding)
- Another one good example is that a person at the same time can have different characteristics. A man act as a father, son, husband and an employee.

Polymorphism



Imagesrc:csharp-station.com

Program for Polymorphism

```
class MultiplyFun
{
    static int Multiply(int a, int b)
    {
        return a * b;
    }
    static double Multiply(double a, double b)
    {
        return a * b;
    }
}
class Main5
{
    public static void main(String[] args)
    {
        System.out.println(MultiplyFun.Multiply(2, 4));
        System.out.println(MultiplyFun.Multiply(5.5, 6.3));
    }
}
```

Output of Polymorphism

```
C:\Users\admin\Desktop\dhilipjava>javac Main5.java
```

```
C:\Users\admin\Desktop\dhilipjava>java Main5
```

```
8
```

```
34.65
```




What is JAVA

- ❖ Java is a high level, class-based object-oriented programming language.
- ❖ It is a high-level programming language which right once and run anywhere.
- ❖ When a programmer writes a java application, then compiled code and he can run that on most operating system (OS) like windows, Linux and MAC OS.
- ❖ It is used to build large enterprise class application.
- ❖ It is used for building large scale system. It is a software only platform running on top of hardware-based platform.
 - ❖ Java Virtual Machine (JVM)
 - ❖ Java Application Programming Interface (Java API)



Characteristics of JAVA

SIMPLE

It is easy for specialized programmer to learn and use.

SECURE

It is providing security while creating internet application

PORTABLE

It can be executed in any environment where java should be there

OBJECT ORIENTED

It is clean, usable, pragmatic approach to objects, not restricted by the need for compatibility with other languages.



Characteristics of JAVA

ROBUST

Java encourage error free programming by doing runtime checking

MULTITHREADED

Java support multithreaded programming

ARCHITECTURAL NEUTRAL

It is not knotted to a specific machine or operating system architecture.

INTERPRETED

Using java bytecode, it supports cross platform code



Characteristics of JAVA

- **HIGH PERFORMANCE**

The Java enables high performance using just in time compiler

- **DISTRIBUTED**

Java was designed for distributed environment of the Internet

- **DYNAMIC**

Java is more dynamic than c and c++, as it is designed to adapt the evolving environment.

Java programs carry substantial amounts of run-time type information that is used to verify and resolve accesses to objects at run time.



OOP in JAVA

Java defines eight simple data types:

- 1) byte-8-bit integer type
- 2) short-16-bit integer type
- 3) int-32-bit integer type
- 4) long-64-bit integer type
- 5) float-32-bit floating-point type
- 6) double-64-bit floating-point type
- 7) char-symbols in a character set
- 8) boolean-logical values

Data Types

DATA TYPES	RANGE	EXAMPLE	USAGE
byte-8-bit integer type	-128 TO 127	Byte b=-15	When working with data streams
short-16-bit integer type	-32768 to 32767	short c= 1000	Least used simple type
int-32-bit integer type	-2147486648 to 2147483647	int b=-600000	It is common one, used to control loops and to index array
long:64-bit integer type	-9223372036854775808 to 9223372036854775807.	long d = 506000000000000078;	When int type is not large enough to hold values

Data Types

float-32-bit floating-point type	$1.4e^{-045}$ to $3.4e^{+038}$	Float d=3.5	Fractional part is used and large degree of accuracy is not needed
double-64-bit floating-point type	$4.9e^{-324}$ to $1.8e^{+308}$	double pi=3.14	Handling large valued number
char-symbols in a character set	0 to 65536	Char a='a'	It is used to represent both ASCII and Unicode character set
Boolean-logical values	True or false	boolean b=(2<3)	It returns relational operators such as 1<2



Variables

It uses variable to store data.

TO allocate memory JVM requires to specify the datatype of the variable.

To associate an identifier of the variable.

It is assigned with initial value.

All are processed as a part of variable declaration.

Variables

declaration	It is used to assign the type of variables
initialization	How to give starting value of variable
scope	How the variables are visible to other parts of the program
lifetime	How the variable is created used and destroyed
type conversion	How automatic type conversion is handled in java
type casting	How variable type is narrowed down



Variable Declaration With Example

Syntax

`datatype identifier=value;`

Example

`int a=20;`

`float i=20.5;`

`double pi=3.14;`

Arrays

- ❖ Array is a collection of related data of similar type.
- ❖ The values can be stored in large amount based on the desire of the user.
- ❖ Arrays can be declared, created, initialized and used.
- ❖ Array is of two types as are shown below
 - One dimensional array
 - Multi dimensional array

Array Initialization

Arrays can be initialized when they are declared:

```
int monthDays[] = {31,28,31,30,31,30,31,31,30,31,30,31};
```

Note:

There is no need to use the new operator

There array is created large enough to hold all specified elements

Array Syntax

Two types of declaring array are as follows

```
type array_variable[];
```

```
type [] array_varriable;
```

After creating an array it does not actually exist. In order to create an array “new” operator should be used.

- `array_variable = new type[size]`

This create new variable to hold size elements of type, which reference will be kept in array variable. Here array variable index starts with zero. Java run time system makes sure that all array indexes are in correct range otherwise it leads to runtime error



Single Dimensional array

Example

```
int days[]={1,2,3,4,5,6,};
```

This is direct way of giving values to array variable. And in this no need to specify the size of the array as we will do in c or c++.

In array concept, if more than one set bracket is there we will call it as multidimensional array. It is also called as array of array.



Multidimensional Array

`int array[][]; //Declaration`

`int array=new int[4][5]; //Creation`

This represents 4 rows and 5 columns.

`int array[][]={{2,3,4},{5,6,7}}// initialization`

This array represents 2 rows and 3 columns



Operator Types

Java operators are used to build value expression.

The listed below are some of the operators used in java

- Arithmetic Operator
- Logical Operator
- Assignment Operator
- Relational Operator
- Bitwise Operator

Arithmetic Operator

Operator	Purpose	a=10,b=3	a=12.5,b=2.0	a='P' b='T'
+	Addition	13 (a+b)	14.5	164
-	Subtraction	7 (a-b)	10.5	-4
*	Multiplication	30 (a*b)	25.0	6720
/	Division	3 (a/b)	6.25	0
%	Modulo Division	1 (a%b)	----	80

Example

a=5 and b=-2 then a%b=1

a=-5 and b=2 then a%b=-1



Logical Operators

LOGICAL OPERATORS :(also called logical connectives)

1. && (logical and)
2. || (logical or)

These 2 operators are used to combine the individual logical expressions into more complex conditions

Results of logical and (&&)

- The result of the logical and (&&) operation will be true only if both operands are true.

Expression 1	Expression2	Result
1	1	1
1	0	0
0	1	0
0	0	0

Results of logical or (||)

- The result of the logical or (||) operation will be true if either operand is true.

Expression 1	Expression 2	Result
1	1	1
1	0	1
0	1	1
0	0	0



Logical Not (!)

- The logical not (!) operator requires only one operand which represents a logical value 0 or non zero.
- The **result** of the operation is **non zero** if and only if the **operand is zero**.
- **Example**
 - $i=7$, and $!(i>6)$ then result is 0.

ASSIGNMENT OPERATORS:

- ✓ Assignment operators are used to assign a value in a particular variable.
- ✓ The most commonly used assignment operator is '='

Assignment operator form

Identifier= expression

- ✓ Identifier may be variable
- ✓ Expression may be constant, variable, or a more complex expression
- ✓ Assignment expressions are referred to as an assignment statement.
- ✓ Ex: a=5;

Multiple assignment

- Multiple assignment of the form:
- **Identifier1 =Identifier2=expression**
- Is equivalent to
- **Identifier1 = (Identifier2=expression)**
- **Example**
- **a=b=5;** It means first 5 is assigned to 'b' and then the value of b (that is 5) is assigned to 'a'.



SHORTHAND ASSIGNMENT

SHORTHAND ASSIGNMENT OPERATORS:

Syntax:

Var operator=exp;

1) += 2) -= 3) *=, 4) /=, 5) %= these are the shorthand operators.

Example

var +=exp is equivalent to var=var+exp

var-=exp is equivalent to var=var-exp

For Ex:

If i=5,j=7

j*=(i-3)

j=j*(i-3) then j=7*(5-3) then j=7*2 finally j=14.

i%=j

i=i%j then i=5%7 then i=5



RELATIONAL OPERATOR:

There are 4 relational operators

- ❖ < (Less than)
- ❖ > (Greater than)
- ❖ <= (Less than or equal to)
- ❖ >= (Greater than or equal to)

There are 2 Equality operators

- ❖ == (equal to)
- ❖ != (not equal to)

➤ These 6 operators are used to form logical expressions, which represent conditions that are either true or false.

RELATIONAL OPERATOR:

- The resulting expressions will be of type integer, since true is represented by integer value 1 and false is represented by the value 0.

Example

$i=1, j=2, k=3$

Expression	Result
$i < j$	1
$(i+j) \geq k$	1
$k \neq 3$	0

BITWISE OPERATORS:

Bitwise operator used for manipulation of data at bit level.

OPERATOR	MEANING
&	Bitwise AND
	Bitwise OR
^	Bitwise Exclusive OR
<<	Shift left
>>	Shift Right
~	One's complement

Expression

The data type of the value returned by an expression depends on the elements used in the expression. The expression

- `number = 0;`

which returns an int because the assignment operator returns a value of the same data type as its left-hand operand; in this case, number is an int.

In Java, compound expressions also be used. The data type required by one part of the expression matches the data type of the other. The example of a compound expression is as follows

- `1 * 2 * 3`

Control statements

- ❖ A program is a sequence of instructions.
- ❖ Java control statements cause the flow of execution to advance and branch based on the changes to the state of the program.
- ❖ Each instruction is executed only once in the order it appears.
- ❖ Control statements are divided into three groups:
 1. Selection statements
 2. Iteration statements
 3. Jump statements



Selection Statements

One of the several possible branches will then be carried out, depending on the outcome of the logical test. This is known as selection statement

- if
- if-else
- nested if-else
- switch

if Statement

- In if statement, only one condition is checked.
- If the condition is true the statement is executed. If the condition is false it will exit out of the loop

```
class IfStatement
{
    public static void main(String[] args)
    {
        int number = 10;
        // checks if number is greater than 0
        if (number > 0)
        {
            System.out.println("The number is positive.");
        }
        System.out.println("Statement outside if block");
    }
}
```

Output

```
C:\Users\admin\Desktop\dhilipjava>javac IfStatement.java
```

```
C:\Users\admin\Desktop\dhilipjava>java IfStatement
```

```
The number is positive.
```

```
Statement outside if block
```


If else Statement

- In this if else statement, if the condition is true then, the true statement was executed or else the else part is executed.
- Or else the statement outside the block was executed.

```
class ifelsestmt
{
    public static void main(String[] args)
    {
        int number = 10;
        // checks if number is greater than 0
        if (number > 0)
        {
            System.out.println("The number is positive.");
        }
        // execute this block
        // if number is not greater than 0
        else
        {
            System.out.println("The number is not positive.");
        }
        System.out.println("Statement outside if...else block");
    }
}
```

Output

```
C:\Users\admin\Desktop\dhilipjava>javac ifelsestmt.java  
  
C:\Users\admin\Desktop\dhilipjava>java ifelsestmt  
The number is positive.  
Statement outside if...else block
```

Switch Statement

- The switch statement allows us to execute a block of code among many alternatives.

```
class switchdemo
{
    public static void main(String[] args)
    {
        int number = 44;
        String size;
        // switch statement to check size
        switch (number)
        {
            case 29:
                size = "Small";
                break;
            case 42:
                size = "Medium";
                break;
            // match the value of week
            case 44:
                size = "Large";
                break;
            // ...
        }
    }
}
```

```
        case 48:
            size = "Extra Large";
            break;
    }
    System.out.println("Size: " + size);
}
```

Output

```
C:\Users\admin\Desktop\dhilipjava>javac switchdemo.java
```

```
C:\Users\admin\Desktop\dhilipjava>java switchdemo
```

```
Size: Large
```

ITERATION STATEMENT

During looping, a set of statements are executed until some conditions for termination of the loop is encountered.

A program loop therefore consists of two segments one known as body of the loop and other is the control statement.

The control statement tests certain conditions and then directs the repeated execution of the statements contained in the body of the loop.

Java provides three iteration statements

- while
- do -while
- for



Jump statements

It enables transfer of control to other parts of the program. In java we are having three types of jump statements

- break
- continue
- return



TYPE CONVERSION

- Conversion in java is done in two ways
 1. Implicit type conversion- Carried out by compiler automatically
 2. Explicit type conversion- Carried out by program using casting
- Size Direction of Data Type
 - - Widening Type Conversion (Casting down)
Smaller Data Type -> Larger Data Type
 - - Narrowing Type Conversion (Casting up)
Larger Data Type -> Smaller Data Type



TYPE CASTING

It is the of assigning a primitive data type value to another data type value. It is also a casting of a class or interface into another class or interface.

Syntax

(target type) values

Example

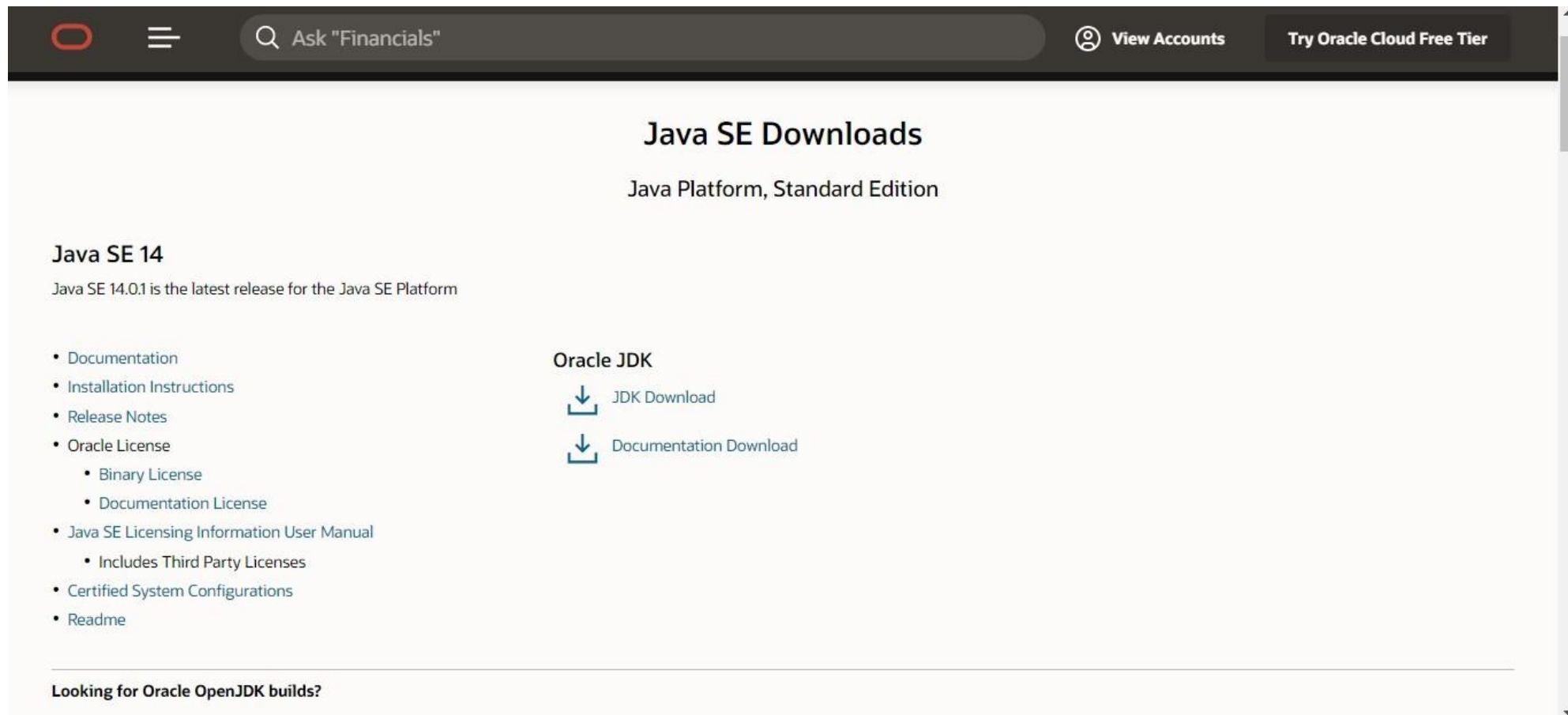
- integer value will be reduced module bytes range:
- int a;
- byte a = (byte) i;
- floating-point value will be truncated to integer value:
 - float a;
 - int b = (int) a;



How to download and install Java for 64-bit machine

The following steps can be followed in order to download and install java. All the steps are described below has been performed on the Windows 10 operating system, but the procedure is quite similar to other operating systems as well.

Step 1: Open <https://www.oracle.com/java/technologies/javase-downloads.html> url in the browser and it will navigate to the official Oracle Java downloads page.

The screenshot shows the Oracle Java SE Downloads page. The header includes the Oracle logo, a search bar with the text 'Ask "Financials"', and links for 'View Accounts' and 'Try Oracle Cloud Free Tier'. The main heading is 'Java SE Downloads' with the subtitle 'Java Platform, Standard Edition'. Under 'Java SE 14', it states 'Java SE 14.0.1 is the latest release for the Java SE Platform'. A list of links on the left includes 'Documentation', 'Installation Instructions', 'Release Notes', 'Oracle License' (with sub-links for 'Binary License' and 'Documentation License'), 'Java SE Licensing Information User Manual' (with a sub-link for 'Includes Third Party Licenses'), 'Certified System Configurations', and 'Readme'. On the right, under 'Oracle JDK', there are two download links: 'JDK Download' and 'Documentation Download', each with a download icon. At the bottom, there is a link for 'Looking for Oracle OpenJDK builds?'.

Step 2: Now, scroll to the version of the Java which we want to download and click on **JDK Download** option as shown below:



The screenshot shows the Oracle Java SE Downloads page. The header includes the Oracle logo, a search bar with "Ask 'Financials'", and links for "View Accounts" and "Try Oracle Cloud Free Tier". The main heading is "Java SE Downloads" with the subtitle "Java Platform, Standard Edition". On the left, a red box highlights the "Java SE 14" section, which states "Java SE 14.0.1 is the latest release for the Java SE Platform". Below this is a list of links: Documentation, Installation Instructions, Release Notes, Oracle License (with sub-links for Binary License and Documentation License), Java SE Licensing Information User Manual (with a sub-link for Includes Third Party Licenses), Certified System Configurations, and Readme. In the center, under the "Oracle JDK" heading, a green box highlights the "JDK Download" link, with a "Documentation Download" link below it. At the bottom, there is a link for "Looking for Oracle OpenJDK builds?".

Java SE Downloads

Java Platform, Standard Edition

Java SE 14

Java SE 14.0.1 is the latest release for the Java SE Platform

- Documentation
- Installation Instructions
- Release Notes
- Oracle License
 - Binary License
 - Documentation License
- Java SE Licensing Information User Manual
 - Includes Third Party Licenses
- Certified System Configurations
- Readme

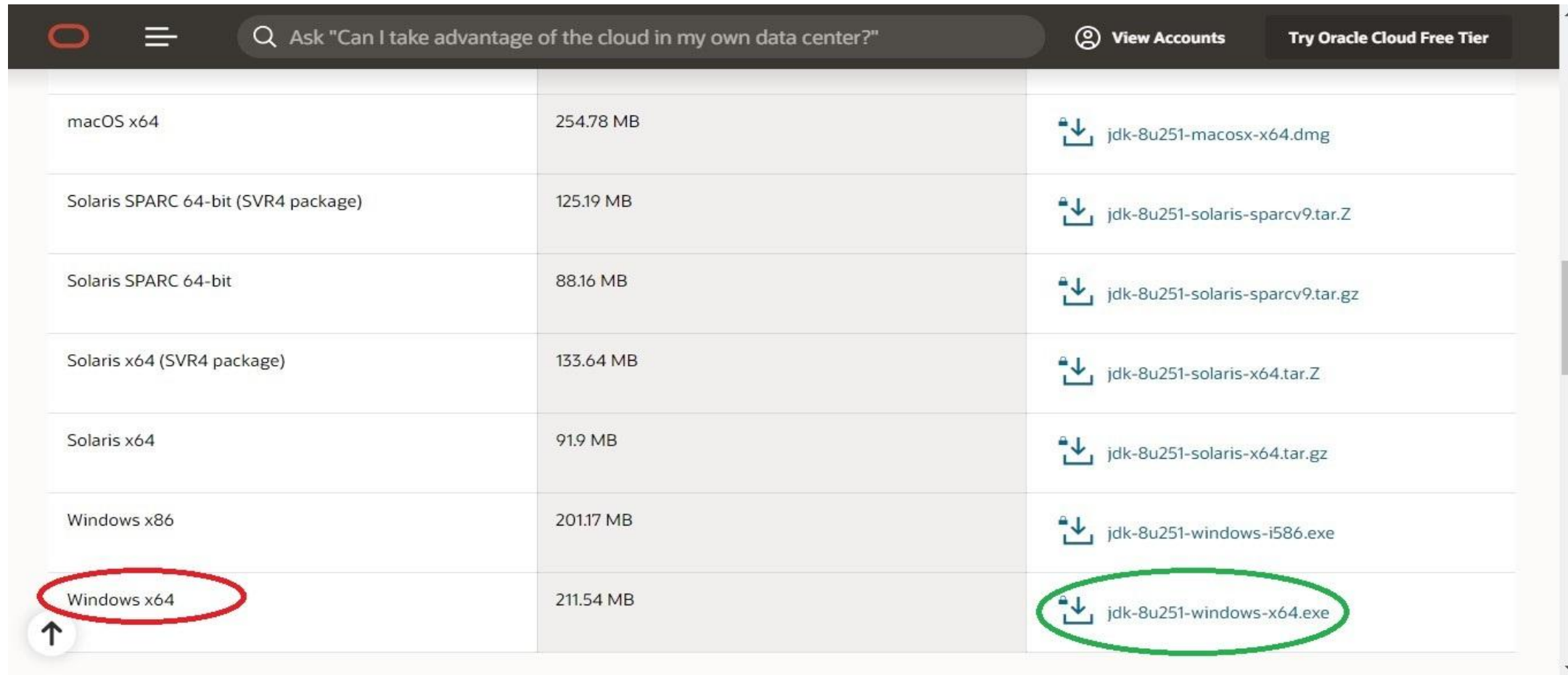
Oracle JDK

JDK Download

Documentation Download

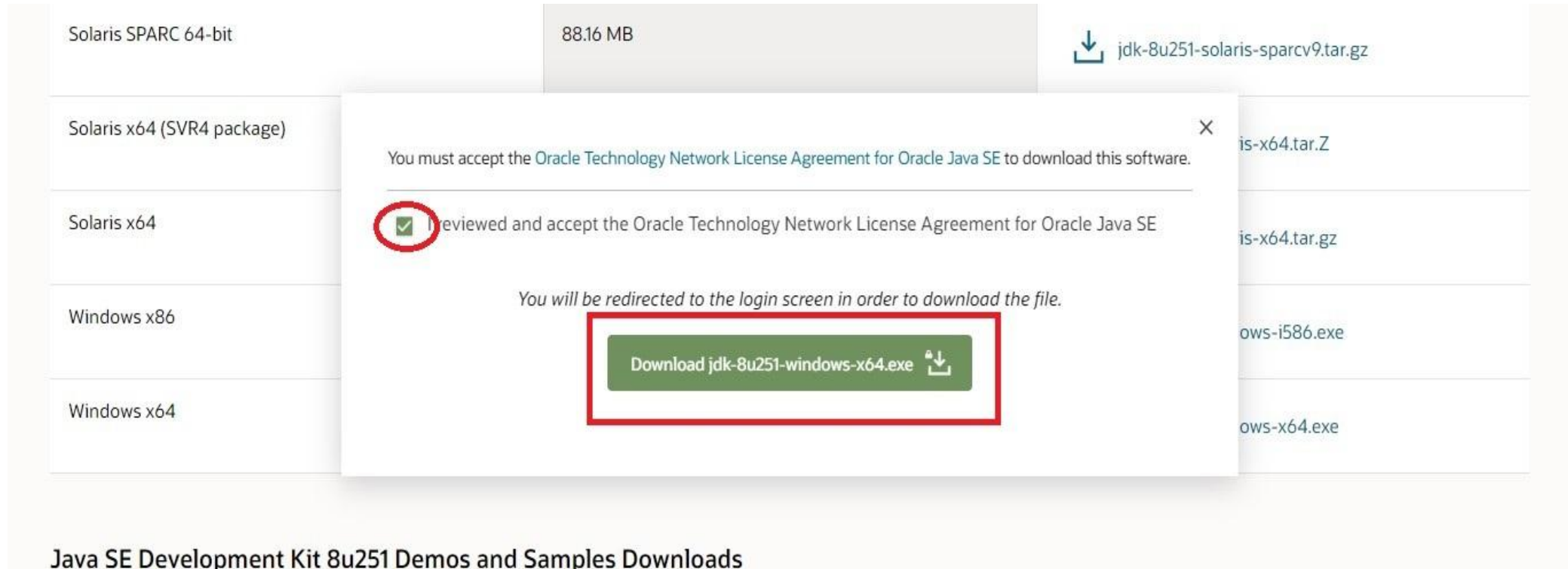
Looking for Oracle OpenJDK builds?

Step 3: Scroll down to the page and click on the download button option suitable for your computer Operating system. But for a 64-bit machine, choose the software name ending with **x64**.

The screenshot shows the Oracle JDK 8u251 download page. The page has a dark header with the Oracle logo, a search bar containing the text 'Ask "Can I take advantage of the cloud in my own data center?"', and links for 'View Accounts' and 'Try Oracle Cloud Free Tier'. Below the header is a table with three columns: Operating System, Size, and Download Link. The table lists various operating systems, including macOS x64, Solaris SPARC 64-bit (SVR4 package), Solaris SPARC 64-bit, Solaris x64 (SVR4 package), Solaris x64, Windows x86, and Windows x64. The 'Windows x64' row is circled in red, and its corresponding download link 'jdk-8u251-windows-x64.exe' is circled in green. An upward arrow icon is located at the bottom left of the table.

Operating System	Size	Download Link
macOS x64	254.78 MB	 jdk-8u251-macosx-x64.dmg
Solaris SPARC 64-bit (SVR4 package)	125.19 MB	 jdk-8u251-solaris-sparcv9.tar.Z
Solaris SPARC 64-bit	88.16 MB	 jdk-8u251-solaris-sparcv9.tar.gz
Solaris x64 (SVR4 package)	133.64 MB	 jdk-8u251-solaris-x64.tar.Z
Solaris x64	91.9 MB	 jdk-8u251-solaris-x64.tar.gz
Windows x86	201.17 MB	 jdk-8u251-windows-i586.exe
Windows x64	211.54 MB	 jdk-8u251-windows-x64.exe

After clicking on the download button, a popup will appear which says that we have to accept **Oracle Technology Network License Agreement for Oracle Java SE** in order to download this software. Therefore, click on the checkbox and then proceed to download as shown below:

The screenshot shows the Oracle Java SE download page. A modal window is open in the center, displaying the 'Oracle Technology Network License Agreement for Oracle Java SE'. The checkbox for 'I reviewed and accept the Oracle Technology Network License Agreement for Oracle Java SE' is checked and circled in red. Below the checkbox, a message states 'You will be redirected to the login screen in order to download the file.' A green button labeled 'Download jdk-8u251-windows-x64.exe' with a download icon is highlighted with a red rectangle. In the background, a list of download links is visible, including 'jdk-8u251-solaris-sparcv9.tar.gz', 'jdk-8u251-windows-x64.tar.Z', 'jdk-8u251-windows-x64.tar.gz', 'jdk-8u251-windows-i586.exe', and 'jdk-8u251-windows-x64.exe'. The page title at the bottom is 'Java SE Development Kit 8u251 Demos and Samples Downloads'.

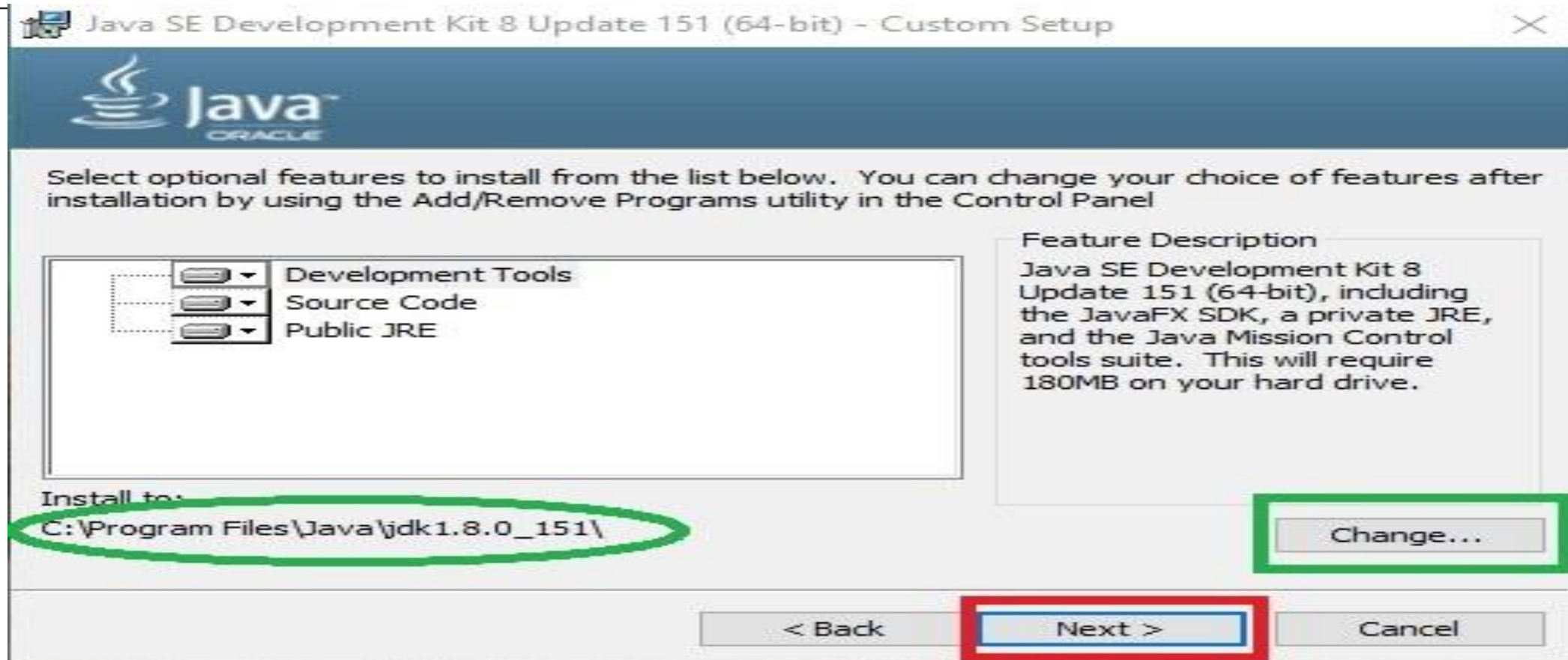
Step 4: We will now be navigated to **Oracle Login page**. We need to login to the account. As soon we log in, our download will start instantly as shown below:



Step 5: After the downloading procedure is complete, we need to run the installer. Once Java installation wizard opens, click on the **Next** button as shown below:



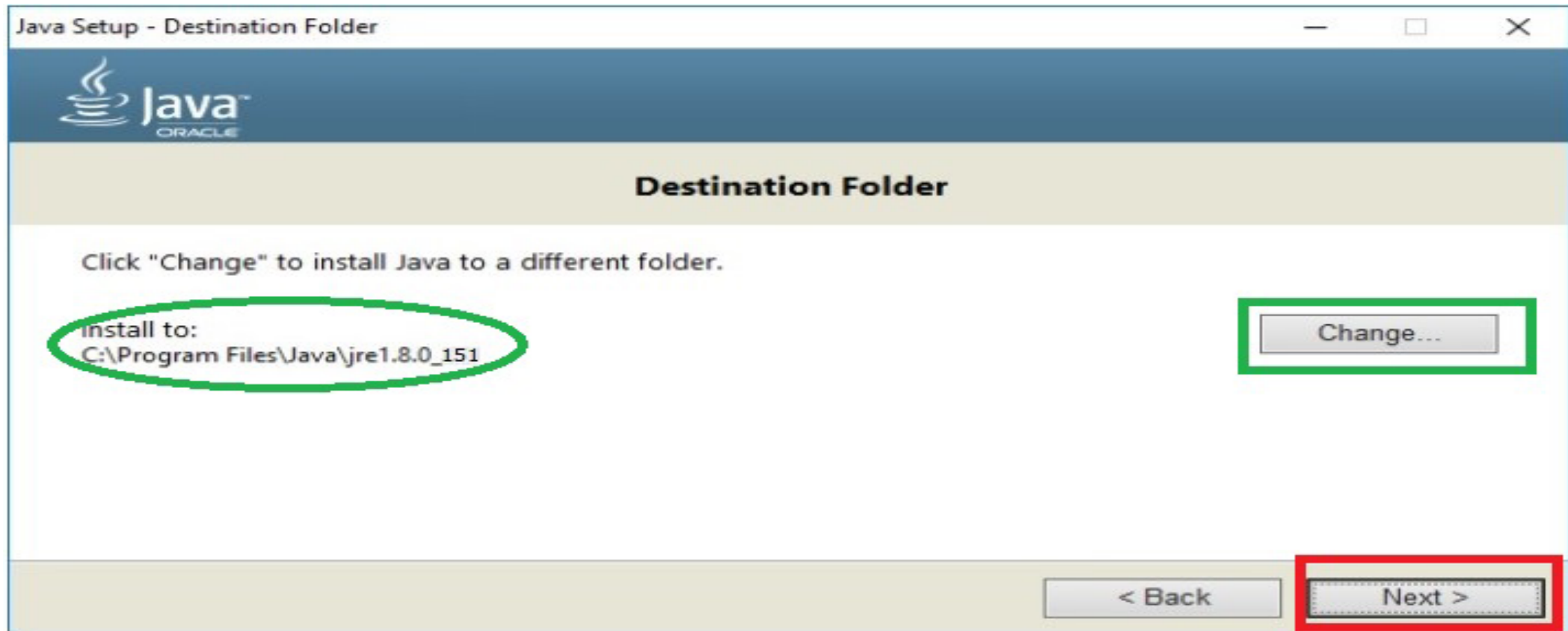
Step 6: Again click on the **Next** button if we wish to install Java development kit in the default directory(encircled with green color), or we can change this directory by clicking on **Change** button.



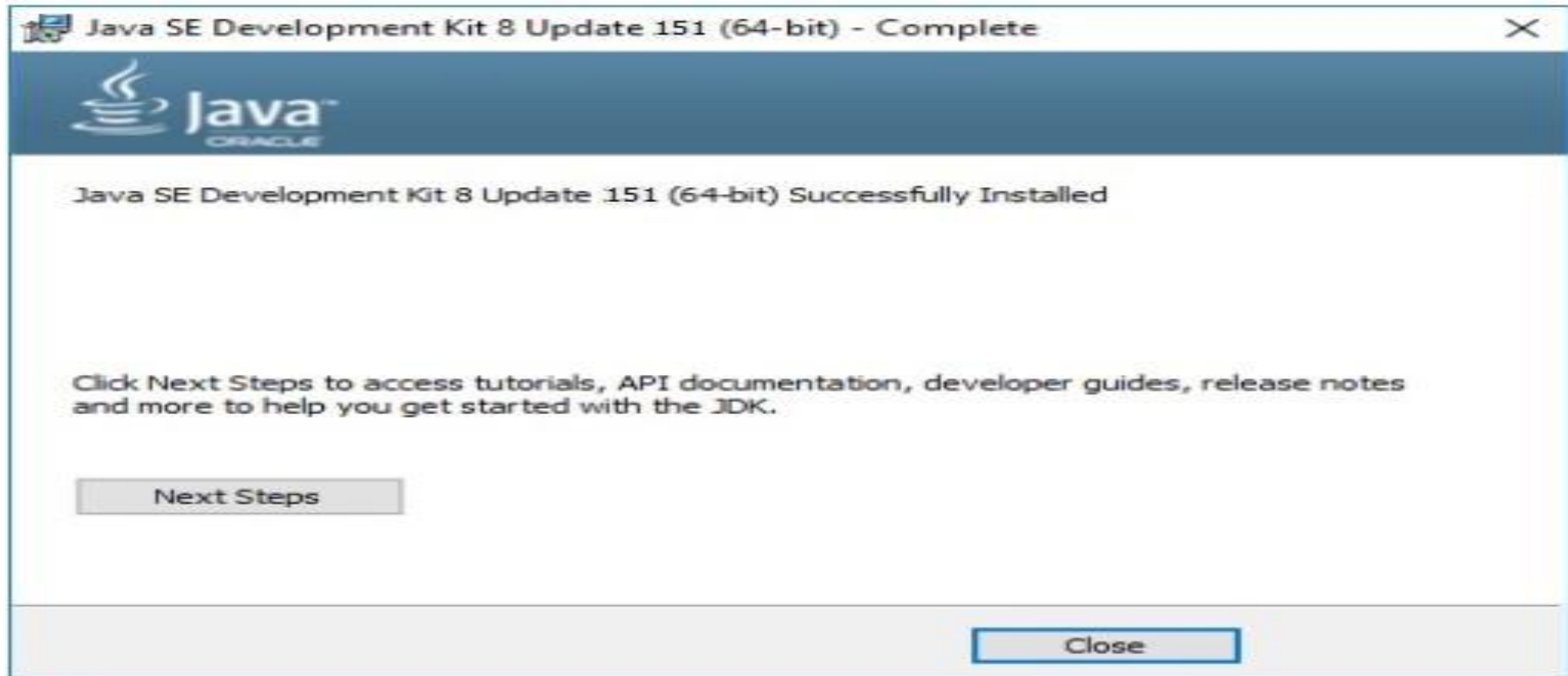
Step 7: The installation will begin as shown below:



Step 8: Now, it will ask for the installation directory for **JRE(Java Runtime Environment)**. Again we can continue with the default directory or change it accordingly.



Step 10: Finally, we can click on the **Close** button after the confirmation window appears that the Java is installed.



Simple Java Program

```
class myfirstjavapgm
{
    public static void main(String[] args)
    {
        System.out.println("First Java program.");
    }
}
```

Type the above program in notepad and save using the class name.java.This file is saved as MyFirstJavaProgram.java



Compiling and running a java program

- Compiling and running a java program is very easy after JDK installation. Following are the steps
- Open a command prompt window and go to the directory where you saved the java program (myfirstjavapgm.java). Assume it in Desktop:\dhilipjava.
- Next, set path for your jdk file. Ex: C:\Users\admin\Desktop\dhilipjava
- Type 'javac myfirstjavapgm.java' and press enter to compile your code. If there are no errors in your code, the command prompt will take you to the next line (Assumption: The path variable is set).
- Now, type ' java myfirstjavapgm ' to run your program.
- You will be able to see the result printed on the window.

Output Screen

```
C:\Users\admin\Desktop\dhilipjava>javac myfirstjavapgm.java  
  
C:\Users\admin\Desktop\dhilipjava>java myfirstjavapgm  
First Java program.
```



JAVA Working Environment

Three Components

- JDK-Java Development Kit
- JVM-Java Virtual Machine
- JRE-Java Runtime Environment



JDK

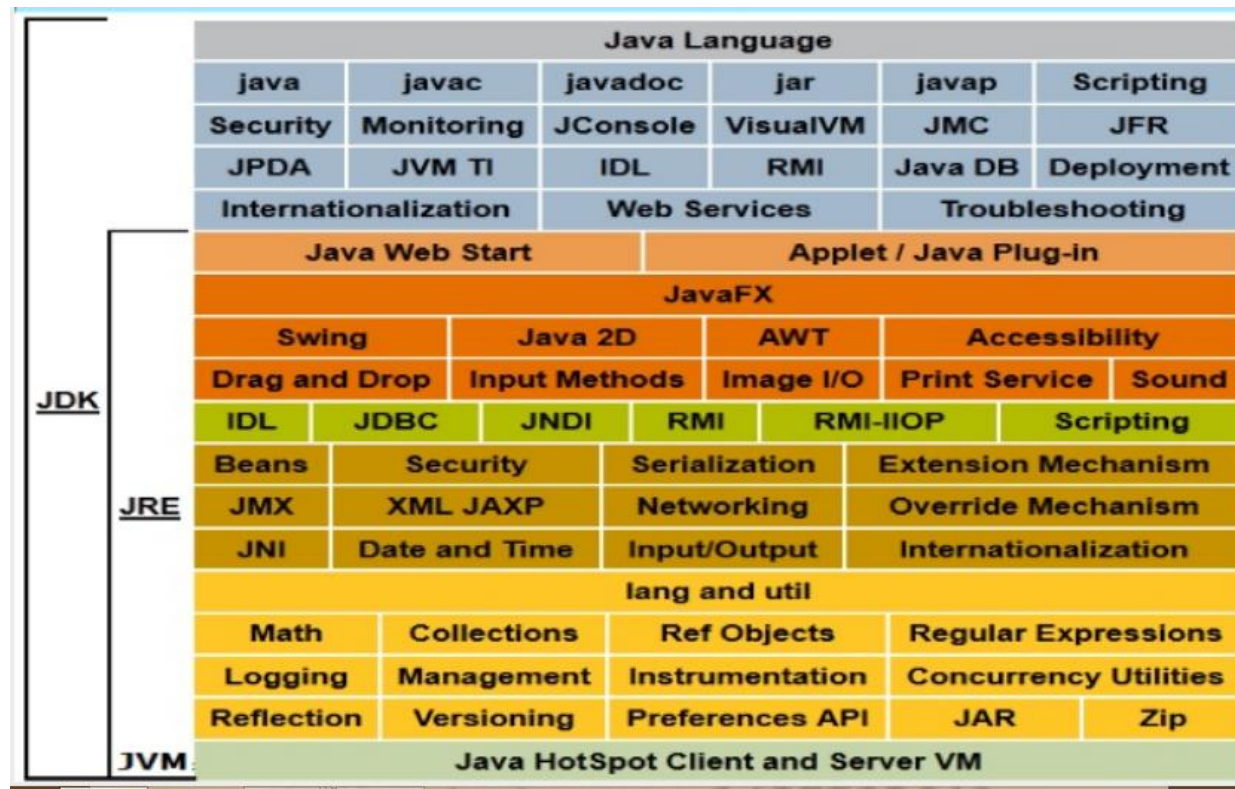
The Java Development Kit (JDK) is primary components.

It physically exists.

It is collection of programming tools and JRE, JVM.

Writing Java applets and applications need development tools like JDK.

JDK Architecture



Source:ictacademy.com



JRE

JRE is an acronym for Java Runtime Environment.

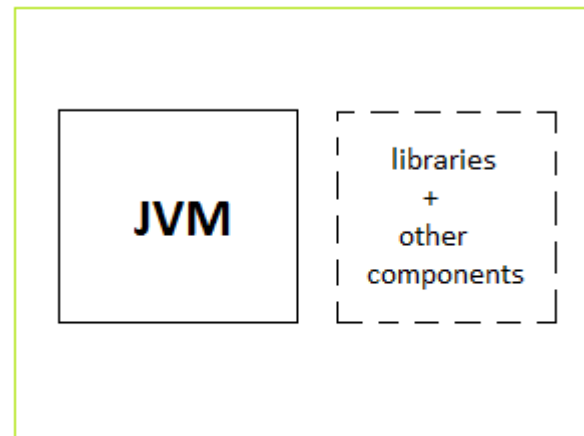
It is also written as Java RTE.

The Java Runtime Environment is a set of software tools.

It physically exists.

It contains a set of libraries + other files that JVM uses at runtime.

JRE Architecture



JRE - Java Runtime Environment

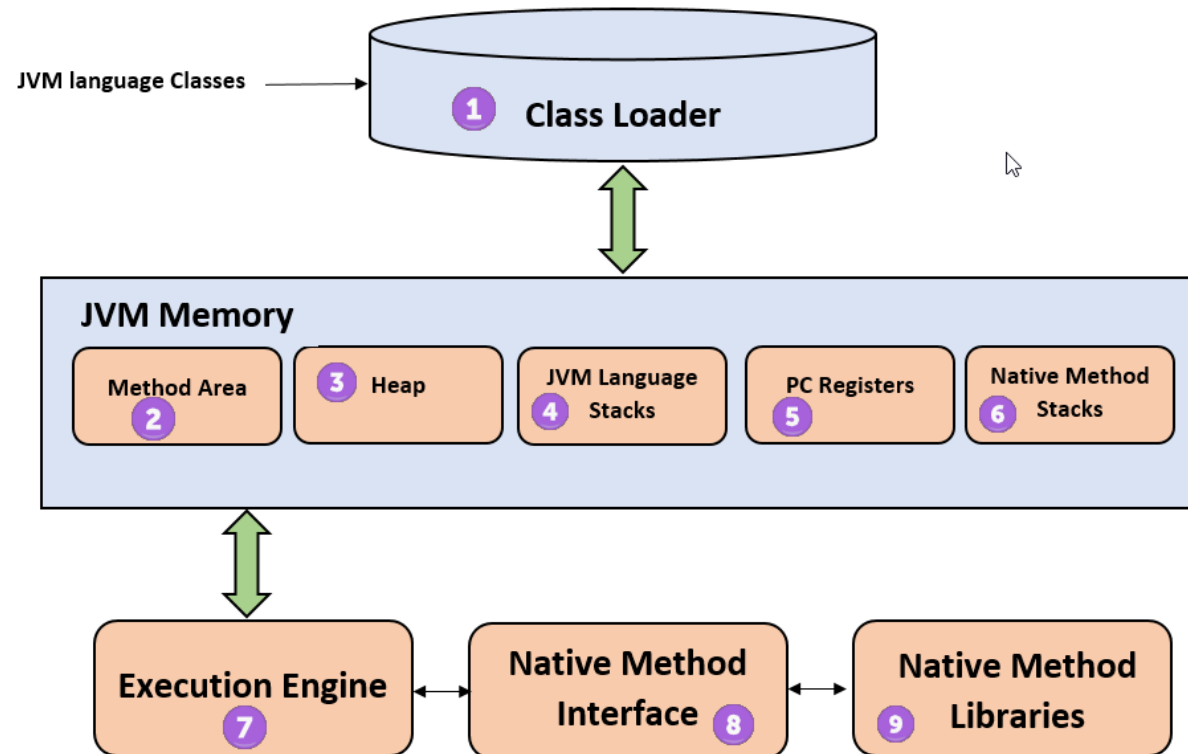
Source: studytonight.com



JVM

- JVM is a engine that provides runtime environment to drive the Java Code or Applications.
- It converts Java byte code into machine languages.
- Java compiler produces code for a virtual machine known as Java Virtual Machine.

JVM Architecture



Source:guru99.com



Uses of JDK,JRE,JVM

- The Java Virtual Machine is built right into your Java software download and helps run Java applications.
- The JRE allows applets written in the Java programming language to run inside various browsers.



Difference between JDK,JRE,JVM

JDK	JRE	JVM
It stands for Java Development Kit.	It stands for Java Runtime Environment	It stands for Java Virtual Machine.
It is the tool necessary to compile, document and package java programs	JRE refers to a runtime environment in which Java bytecode can be executed.	It is an abstract machine. It is a specification that provides a runtime environment in which Java bytecode can be executed.
It contains JRE+development tools.	It's an implementation of the JVM which physical exists.	JVM follow three notations: Specification, Implementation and Runtime Instance

Topic 1

OOP Concepts

Topic 2

Characteristics of JAVA

Topic 3

OOP in JAVA

Topic 4

JAVA Working Environment

First Lesson Summary

Learned about OOP concepts, Characteristics of JAVA, OOP in JAVA, JAVA Working Environment

Fundamental Programming Constructs

Lesson 1. Object Oriented programming Concepts

Lesson 2. Fundamental programming Constructs

Lesson 3. Access Specifiers

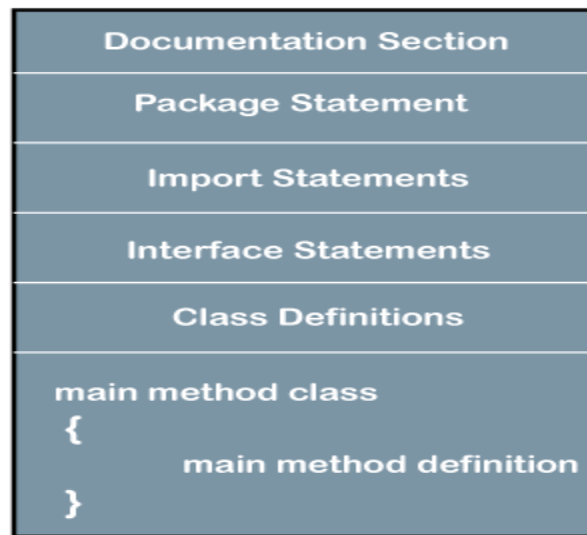
Lesson 4. Constructors

Lesson 5. Inheritance

Introduction



- A program is a sequence of instructions that a computer can execute to perform some task. There are two basic blocks of programming: data and instructions and to work with data, you need to understand variable and data types; to work with instructions, you need to understand control structures and subroutines (methods).



Structure of Java Program

Structure of Java Program

- Java is called as Platform independent, Object oriented, and secure programming language.
- Using the java programming language, we can develop a wide variety of applications. So we want to understand the basic structure of java program in detail.

package details	→	<code>import java.io.*</code>
<code>class className</code>	→	<code>class Sum</code>
{		
Data members;	→	<code>int a, b, c;</code>
user_defined method;	→	<code>void display();</code>
<code>public static void main(String args[])</code>		
{		
Block of Statements;	→	<code>System.out.println("Hello Java !");</code>
}		
}		



Structure of Java Program

- A package is a collection of classes, interfaces and sub-packages. A sub package contains collection of classes, interfaces and sub-sub packages etc. `java.lang.*`; package is imported by default and this package is known as default package.
- Class is keyword used for developing user defined data type and every java program must start with a concept of class. "ClassName" represent a java valid variable name treated as a name of the class each and every class name in java is treated as user-defined data type.
- Data member represents either instance or static they will be selected based on the name of the class.

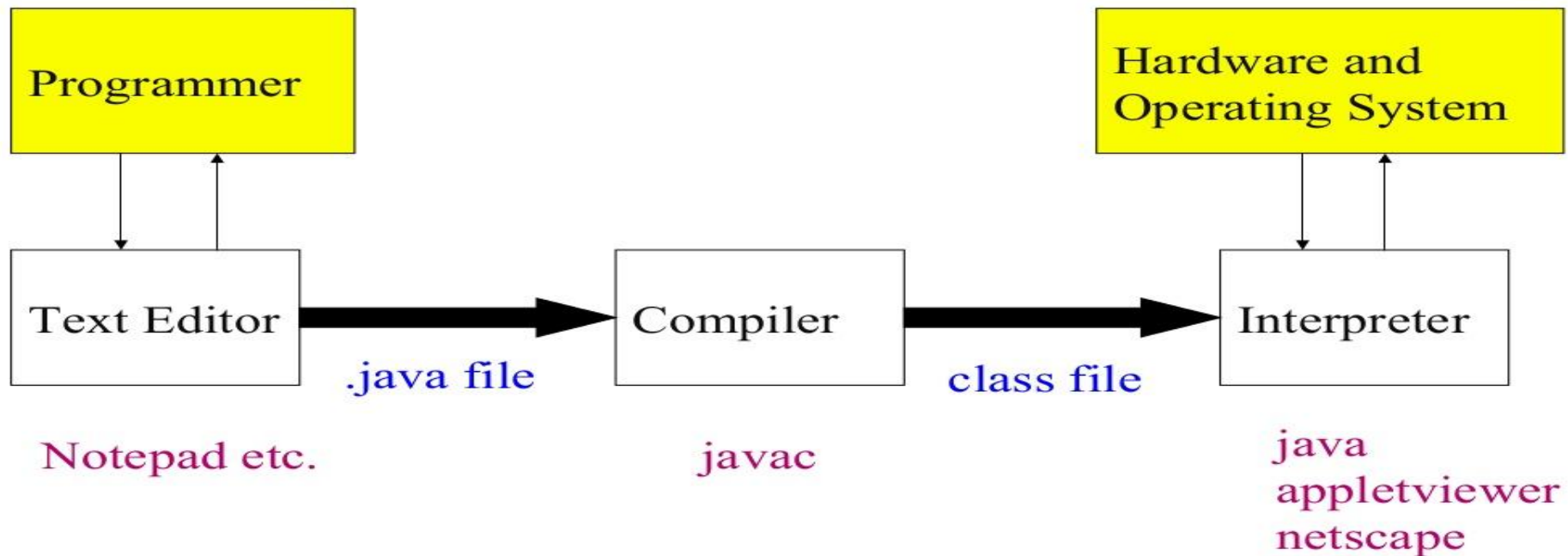
Structure of Java Program

- Each and every java program starts execution from the main() method. And hence main() method is known as program driver.
- Since main() method of java is not returning any value and hence its return type must be void.
- Since main() method of java executes only once throughout the java program execution and hence its nature must be static.
- Since main() method must be accessed by every java programmer and hence whose access specifier must be public.
- Each and every main() method of java must take array of objects of String.
- Block of statements represents set of executable statements which are in term calling user-defined methods are containing business-logic.
- The file naming convention in the java programming is that which-ever class is containing main() method, that class name must be given as a file name with an extension .java.

Structure of Java Program



Java is Compiled and Interpreted



Classes in Java



- . class defines a new data type.
- . Once defined, this new type can be used to create objects of that type.
- . Class is a template for an object, and an object is an instance of a class.
- . A class consists of Data members and methods.
- . The primary purpose of a class is to hold data/information. This is achieved with attributes which are also known as data members.
- . The member functions determine the behavior of the class, i.e. provide a definition for supporting various operations on data held in the form of an object.

Declaring Objects

- When you create a class, you are creating a new data type. You can use this type to declare objects of that type.
- Obtaining objects of a class is a two-step process. First, you must declare a variable of the class type. This variable does not define an object. Instead, it is simply a variable that can refer to an object. Second, you must acquire an actual, physical copy of the object and assign it to that variable. You can do this using the new operator. The new operator dynamically allocates (that is, allocates at run time) memory for an object and returns a reference to it. This reference is, more or less, the address in memory of the object allocated by new. This reference is then stored in the variable. Thus, in Java, all class objects must be dynamically allocated.

```
classname class -var = new classname ( );
```

Java Comments

- . Comments can be used to explain Java code, and to make it more readable. It can also be used to prevent execution when testing alternative code.
- . Single-line comments start with two forward slashes (//).
- . Any text between // and the end of the line is ignored by Java (will not be executed).
- . This example uses a single-line comment before a line of code:

```
// This is a comment  
System.out.println("Hello world")  
/* This is a Simple Java Program  
This is a comment */
```

Data Types



Characters

The char data type is used to store a **single** character. The character must be surrounded by single quotes, like 'A' or 'c'

```
public class Main {  
    public static void main(String[] args) {  
        char myGrade = 'B';  
        System.out.println(myGrade);  
    }  
}
```

Data Types



Strings

- The String data type is used to store a sequence of characters (text). String values must be surrounded by double quotes.

```
public class Main {  
    public static void main(String[] args) {  
        String greeting = "Hello World";  
        System.out.println(greeting);  
    }  
}
```

Variables

- Variables are symbolic names for memory storage.
- Like most compiled languages, variables must be declared before they can be used.
- Variables are a symbolic name given to a memory location.
- All variables have a type which is enforced by the compiler
- The general form of a variable declaration is:

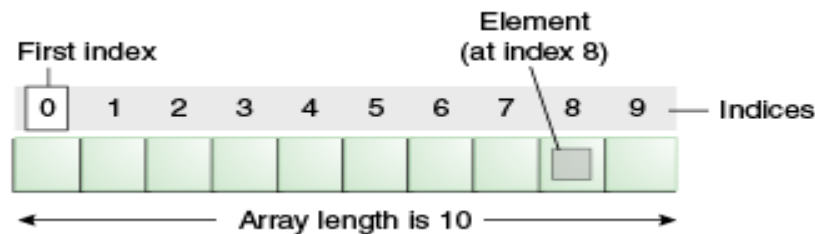
`type variable-name [= value][,variable-name[= value]];`

```
int total;  
float xValue  
= 0.0;  
boolean  
isFinished;
```

`String name;`

Arrays

An *array* is a container object that holds a fixed number of values of a single type. The length of an array is established when the array is created. After creation, its length is fixed. You have seen an example of arrays already, in the main method of the "Hello World!" application.



Each item in an array is called an *element*, and each element is accessed by its numerical *index*. As shown in the preceding illustration, numbering begins with 0. The 9th element, for example, would therefore be accessed at index 8.

Arrays – Example Program

```
class ArrayDemo {  
    public static void main(String[] args) {  
        // declares an array of integers  
  
        int[] anArray; // allocates memory for 10 integers anArray = new int[10]; // initialize first element anArray[0] = 100;  
        // initialize second element anArray[1] = 200; // and so forth  
  
        anArray[2] = 300;  
        anArray[3] = 400;  
        anArray[4] = 500;  
        anArray[5] = 600;  
        anArray[6] = 700;  
        anArray[7] = 800;  
        anArray[8] = 900;  
        anArray[9] = 1000;
```

```
        System.out.println("Element at index 0: " + anArray[0]);
```

Array Declaration & Initialization

The following code snippet shows the two dimensional array declaration in Java Programming Language:

```
Data_Type[][] Array_Name;
```

Data_type: It decides the type of elements it will accept. For example, If we want to store integer values, then the Data Type will be declared as int. If we want to store Float values, then the Data Type will be float.

Array_Name: This is the name to give it to this Java two dimensional array. For example, Car, students, age, marks, department, employees, etc.

```
for ( int i=0; i<n ;i++)  
{  
    for (int j=0; j<n; j++)  
    {  
        a[i][j] = 0;  
    }  
}
```


Array Declaration & Initialization

In the Java programming language, a multidimensional array is an array whose components are themselves arrays. This is unlike arrays in C or Fortran. A consequence of this is that the rows are allowed to vary in length, as shown in the following [MultiDimArrayDemo](#) program:

```
class MultiDimArrayDemo
{
    public static void main(String[] args)
    {
        String[][] names = { {"Mr. ", "Mrs. ", "Ms. "}, {"Smith", "Jones"} };
        // Mr. Smith
        System.out.println(names[0][0] + names[1][0]);
        // Ms. Jones
        System.out.println(names[0][2] + names[1][1]); } }
```

The output from this program is:

Mr. Smith Ms. Jones

Topic 1
Introduction - Structure of Java Program

Topic 2
Defining Class

Topic 2
Data Types

Topic 3
Variables

Topic 4
Operators

Topic 4
Control Flow

Topic 5
Arrays

Summary

LEARN FUNDAMENTAL PROGRAMMING
STRUCTURE FOR WRITING SIMPLE STAND ALONE
APPLICATION USING CONTROL, LOOPING AND
ARRAYS.

Access Specifiers

Lesson 1. Object Oriented programming Concepts

Lesson 2. Fundamental Programming Constructs

Lesson 3: Access Specifiers

Lesson 4. Constructors

Lesson 5. Inheritance

Access Specifiers

There are two types of modifiers in Java:

- access modifiers
- non-access modifiers.

The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class. The different types of access specifiers are as follows

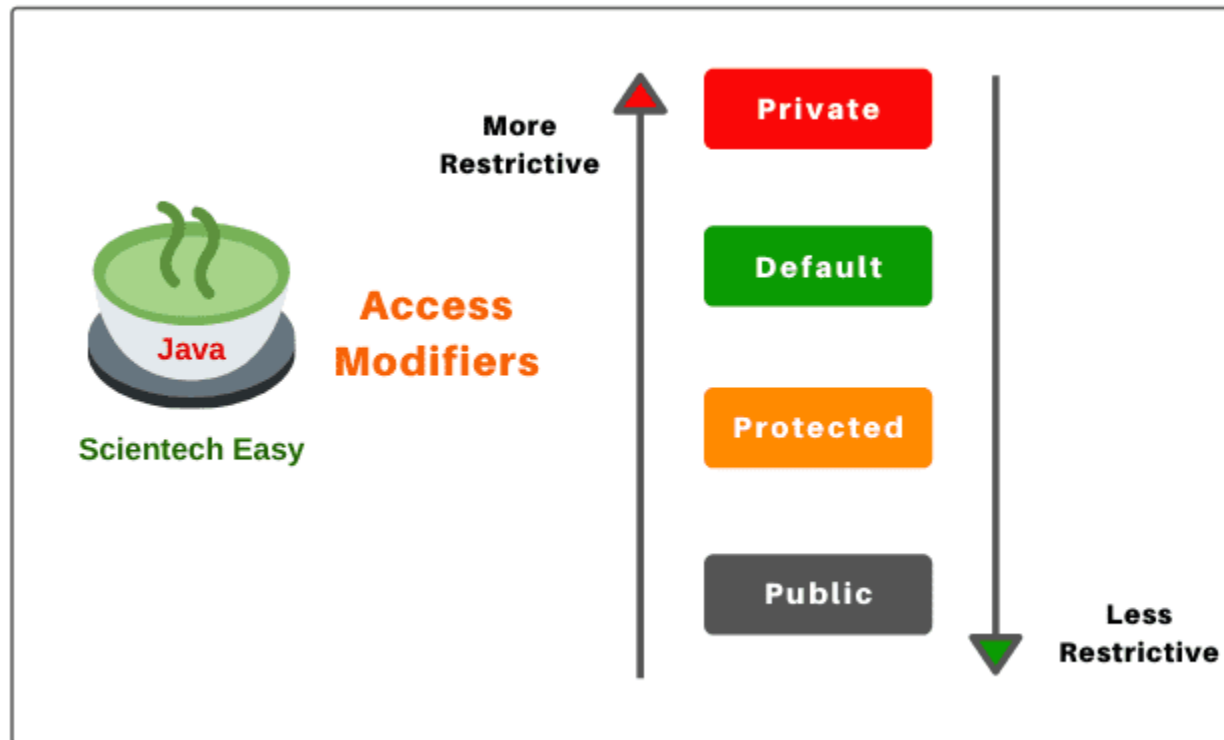
private: When a member of a class is specified as private, then that member can only be accessed by other members of its class.

public: When a member of a class is modified by public, then that member can be accessed by any other code.

protected: applies only when inheritance is involved.

default: same as specifying public.

Access Specifiers



Access Specifiers



	default	private	protected	public
Same Class	Yes	Yes	Yes	Yes
Same package subclass	Yes	No	Yes	Yes
Same package non-subclass	Yes	No	Yes	Yes
Different package subclass	No	No	Yes	Yes
Different package non-subclass	No	No	No	Yes

Private



```
class A
{
    private int data=40;
    private void msg()
    {
        System.out.println("Hello java");
    }
}
public class Simple1
{
    public static void main(String args[])
    {
        A obj=new A();
        System.out.println(obj.data); //Compile Time Error
        obj.msg(); //Compile Time Error
    }
}
```

Output



```
C:\Users\admin\Desktop\dhilipjava>javac Simple1.java
Simple1.java:14: error: data has private access in A
        System.out.println(obj.data); //Compile Time Error
                               ^
Simple1.java:15: error: msg() has private access in A
        obj.msg(); //Compile Time Error
               ^
2 errors
```


Public



Public members of any class are accessible anywhere in the program inside the same class and outside of class, within the same package and outside of the package. The other name of public is universal access specifier

```
class Hello
{
    public int a=20;
    public void show()
    {
        System.out.println("Hello java");
    }
}
public class Demo
{
    public static void main(String args[])
    {
        Hello obj=new Hello();
        System.out.println(obj.a);
        obj.show();
    }
}
```

output



```
C:\Users\admin\Desktop\dhilipjava>javac Demo.java
```

```
C:\Users\admin\Desktop\dhilipjava>java Demo
```

```
20
```

```
Hello java
```

Protected:



Protected members of the class are accessible within the same class and another class of the same package and also accessible in inherited class of another package. Protected are also called derived level access modifiers.

```
package pack1;  
public class B1  
{  
protected void show()  
{  
System.out.println("Hello Java");  
}  
}
```

Protected member Access Example

```
public class Shape
{
    protected double height; // To hold height.    protected double width; // To hold width or base    public void setValues( double height ,
    double width )
    {

this. height = height;    this. width = width ;
    } }

public class Rectangle extends Shape
{
    public double getArea ()
    {
return height * width ; // accessing protected members
    } }

public class TestProgram
{
    public static void main( String [] args)
    {

Rectangle rectangle = new Rectangle ();    rectangle. setValues (5 ,4);
System . out. println (" Area of rectangle : " + rectangle. getArea ());
    } }
```

Static Members



- . Static members are those which belongs to the class and you can access these members without instantiating the class.
- . The static keyword can be used with methods, fields, classes (inner/nested), blocks.
- . **Static Methods** – You can create a static method by using the keyword *static*. Static methods can access only static fields, methods. To access static methods there is no need to instantiate the class, you can do it just using the class name as

Static Member



```
public class MyClass
{
    public static void sample()
    {
        System.out.println("Hello");
    } public static
    void main(String args[])
    {
        MyClass.sample();
    }
}
```

Topic 1
Access Specifiers

Topic 2
Static Members

Third Lesson Summary

From this topic we learned the visibility about the classes and data members.

Constructors



Lesson 1. Object Oriented programming Concepts

Lesson 2. Fundamental programming Constructs

Lesson 3. Access Specifiers

Lesson 4. Constructors

Lesson 5. Inheritance

Constructors



In Java, a constructor is a block of codes similar to the method.

It is called when an instance of the class is created.

At the time of calling constructor, memory for the object is allocated in the memory.

It is a special type of method which is used to initialize the object.

Types of Constructors



A constructor having no parameter is known as default constructor and no-arg constructor.

Constructor having parameter is known as parameterized constructor.

```
class Box {  
    double    width ;  
    double    height;  
    double    depth ;  
  
    // default Constructor  
    Box ()  
    {    width = 3;    height = 4;    depth = 2;    }  
    // parameterized Constructor  
    Box( double w, double h, double d)  
    {    width = w;    height = h;    depth = d;    }  
}
```

Example

```
class BoxDemo7 {  
    public static void main( String args [])  
    {  
        Box mybox1 = new Box();  
        Box mybox2 = new Box(3, 6, 9);  
        double vol;  
        // get volume of first box  
        vol = mybox1 . volume ();   System . out .  
        println ("Volume is " + vol);  
        // get volume of second box  
        vol = mybox2 . volume ();   System . out .  
        println ("Volume is " + vol);  
    }  
}
```

this keyword

this can be used inside any method to refer to the current object. That is, this is always a reference to the object on which the method was invoked.

```
Box( double w, double h, double d) {  
    this.width = w;  
    this.height = h;  
    this.depth = d;  
}
```

Stack Implementation

```
class Stack
{
    int stck [] = new int [10];
    int tos;
    Stack ()
    {
        tos = -1;
    }
    void push(int item)
    {
        if (tos == 9)
            System.out.println("Stack is full.");
        else
            stck[++tos] = item;
    }
    int pop()
    {
        if (tos < 0) {
            System.out.println("underflow");
            Stack return 0;
        }
        else
            return stck[tos--];
    }
}
```

```
class TestStack {  
    public static void main( String args []) {  
        Stack mystack1 =new Stack();  
        Stack mystack2 = new Stack();  
        for( int i=0; i <10; i++) mystack1 . push(i);  
        for(int i=10; i <20; i++) mystack2 . push(i);  
        System . out . println("Stack in mystack1:");  
        for( int i=0; i <10; i++)  
            System . out . println( mystack1 . pop());  
        System . out . println("Stack in mystack2  
        :");  for( int i=0; i <10; i++)  
            System . out . println( mystack2 . pop());  
    }  
}
```

Topic 1
Default Constructor

Topic 2
Parameterized Constructor

Third Lesson Summary

Learnt the uses of constructors to assign initial values to the data members through default and parameterized constructors

Inheritance



Lesson 1. Object Oriented programming Concepts

Lesson 2. Fundamental programming Constructs

Lesson 3. Access Specifiers

Lesson 4. Constructors

Lesson 5. Inheritance



Inheritance Introduction

Definition of Inheritance

Inheritance is the mechanism of deriving new class from old one, old class is known as superclass and new class is known as subclass.

The subclass inherits all of its instances variables and methods defined by the superclass and it also adds its own unique elements.

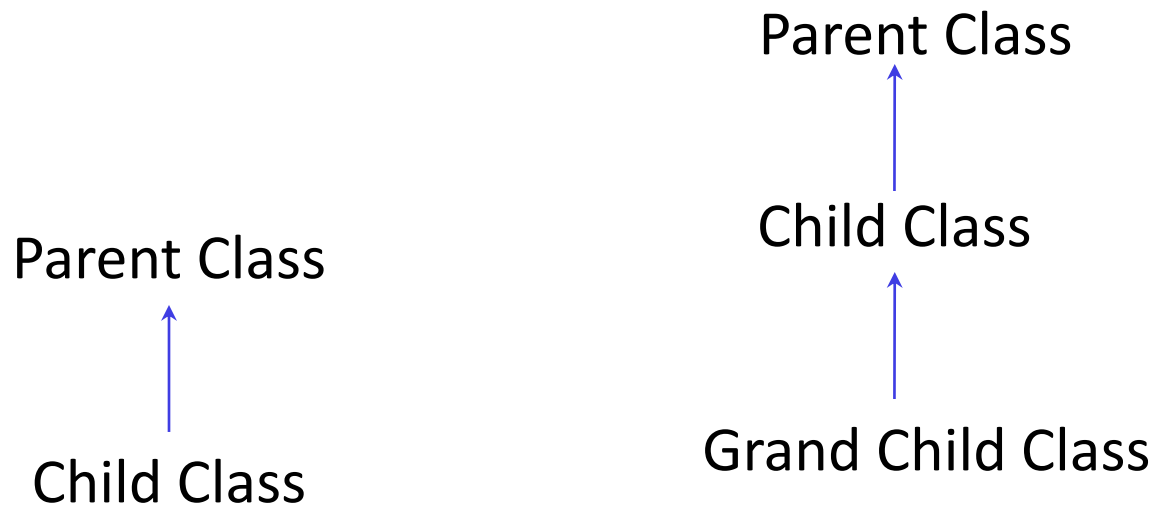
Thus, we can say that subclass is specialized version of superclass.

Syntax

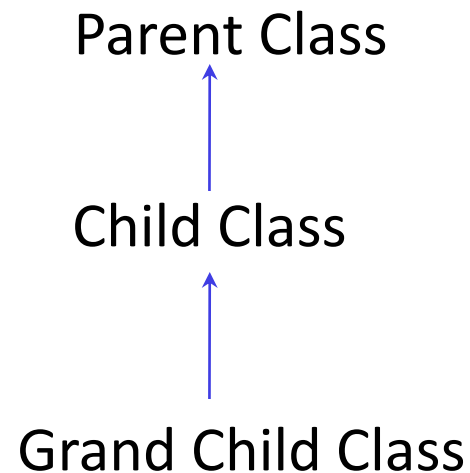
extends is the keyword used to inherit the properties of a class

```
class Super
{ .....
..... }
class Sub extends Super
{ .....
..... }
```

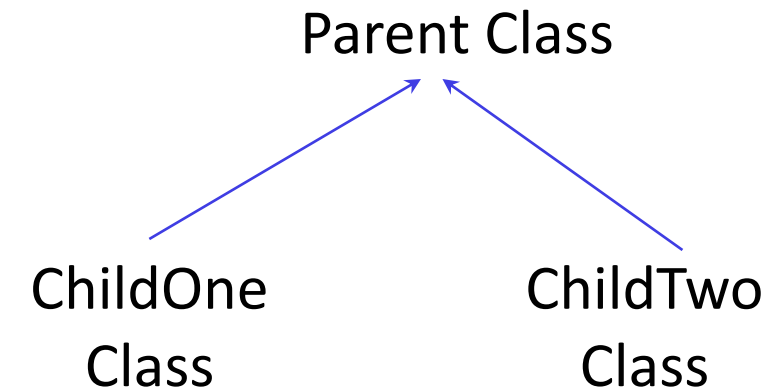
Types of Inheritance



**Single
Inheritance**



**Multilevel
Inheritance**



**Hierarchical
Inheritance**

Single Inheritance

- A class inherits another class
- For example the Dog class inherits the Animal class as shown below

```
class Animal{  
void eat(){System.out.println("eating...");}  
}  
class Dog extends Animal{  
void bark(){System.out.println("barking...");}  
}  
class TestInheritance{  
public static void main(String args[]){  
Dog d=new Dog();  
d.bark();  
d.eat();  
}}}
```

Output
barking...
eating...

Multiple Inheritance

- chain of inheritance
- its like child, parent and grandparent relationship
- for example a BabyDog class inherits the Dog class which again inherits the Animal class

Example Program

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class BabyDog extends Dog{
void weep(){System.out.println("weeping...");}
}
class TestInheritance2{
public static void main(String args[]){
BabyDog d=new BabyDog();
d.weep();
d.bark();
d.eat();
}}
```

Output
weeping...
barking...
eating...

Hierarchical Inheritance

- when two or more classes inherits a single class
- for example Dog class and Cat class inherits the Animal class

Example Program

```
class Animal{  
void eat(){System.out.println("eating...");}  
}  
class Dog extends Animal{  
void bark(){System.out.println("barking...");}  
}  
class Cat extends Animal{  
void meow(){System.out.println("meowing...");}  
}  
class TestInheritance3{  
public static void main(String args[]){  
Cat c=new Cat();  
c.meow();  
c.eat();  
//c.bark();//Compile Time Error  
}}
```

Output
meowing...
eating...



INTERFACE

- An **interface in java** is a blueprint of a class.
- It has static constants and abstract methods only.
- The interface in java is a **mechanism to achieve fully abstraction**.
- There can be only abstract methods in the java interface not method body.
- It is used to achieve fully abstraction and multiple inheritance in Java.

Program on Interface

```
interface IntStack
{
    void push(int item);
    int pop();
}
```

```
class FixedStack implements IntStack
{
    private int stack[];
    private int tos;
    FixedStack(int size)
    {
        stack=new int[size];
        tos=-1;
    }
    public void push(int item)
    {
        if(tos==stack.length-1)
            System.out.println("stack is full");
        else
            stack[++tos]=item;
    }
    public int pop()
    {
        if(tos<0)
        {
            System.out.println("stack underflow");
            return 0;
        }
    }
}
```

```
    }  
    else  
        return stack[tos--];  
}  
public static void main(String args[])  
{  
    FixedStack mystack1=new FixedStack(5);  
    FixedStack mystack2=new FixedStack(8);  
    for(int i=0;i<6;i++) mystack1.push(i);  
    for(int i=0;i<9;i++) mystack2.push(i);  
    System.out.println("stack in mystack1");  
    for(int i=0;i<6;i++)  
        System.out.println(mystack1.pop());  
    System.out.println("stack in mystack2");  
    for(int i=0;i<9;i++)  
        System.out.println(mystack2.pop());  
}
```


Output

```
C:\Users\admin\Desktop\dhilipjava>java FixedStack
stack is full
stack is full
stack in mystack1
4
3
2
1
0
stack underflow
stack in mystack2
7
6
5
4
3
2
1
0
stack underflow
0
```

Topic 1

Inheritance Introduction

Topic 2

Types of Inheritance

Topic 3

Private Member Access

Topic 4

Usage of Super

Topic 5

Protected Member Access

Topic 6

When constructors are called

Lesson Summary

Understood Inheritance and solve real world problem by implementing different types of inheritance

Packages



Lesson 1. Object Oriented programming Concepts

Lesson 2. Fundamental programming Constructs

Lesson 3. Access Specifiers

Lesson 4. Constructors

Lesson 5 : Inheritance

Lesson 6. Packages

Packages in Java



A **java package** is a group of similar types of classes, interfaces and sub-packages.

Package in java can be categorized in two form, built-in package and user-defined package.

There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.

Here, we will have the detailed learning of creating and using user-defined packages.



Packages in Java

Java uses file system directories to store packages. For example, the **.class** files for any classes you declare to be part of **MyPackage** must be stored in a directory called **MyPackage**.

You can create a hierarchy of packages. The general form of a multileveled package statement is shown here: **package pkg1[.pkg2[.pkg3]];**

Example

```
Simple.java - Notepad
File Edit Format View Help

package mypack;
public class Simple{
    public static void main(String args[]){
        System.out.println("Welcome to
package");
    }
}
```

```
Command Prompt
Microsoft Windows [Version 6.3.9600]
(c) 2013 Microsoft Corporation. All rights reserved.

C:\Users\guru>d:
D:\>cd java
D:\Java>set path="C:\Program Files\Java\jdk1.8.0_261\bin"
D:\Java>javac -d . Simple.java
D:\Java>java mypack.Simple
Welcome to package
D:\Java>
```

How to access package from another package?



There are three ways to access the package from outside the package.

- `import package.*;`
- `import package.classname;`
- fully qualified name.

Using packagename.*



Using packagename.*

If you use `package.*` then all the classes and interfaces of this package will be accessible but not sub packages. The `import` keyword is used to make the classes and interface of another package accessible to the current package.

```
package pack;
public class A
{
    public void msg()
    {
        System.out.println("Hello this is using packagename.*");
    }
}
```

```
package mypack;
import pack.*;
class B
{
    public static void main(String args[])
    {
        A obj = new A();
        obj.msg();
    }
}
```


Using packagename.classname



Using packagename.classname

If you import `package.classname` then only declared class of this package will be accessible.

Sample package created

 mypack	5/21/2020 12:19 PM	File folder
 mypack1	5/21/2020 12:24 PM	File folder
 mypack2	5/21/2020 12:33 PM	File folder
 p1	9/11/2020 10:00 AM	File folder
 pack	5/21/2020 12:19 PM	File folder
 pack1	5/21/2020 12:37 PM	File folder

Using fully qualified name



If you use fully qualified name then only declared class of this package will be accessible.

Now there is no need to import.

But you need to use fully qualified name every time when you are accessing the class or interface.

Topic 1

Packages in Java

Topic 2

Importing Packages

Topic 3

Builtin Packages in Java

Sixth Lesson Summary

Learned about Packages in Java, importing Packages, Builtin Packages in Java, JavaDoc, JavaDoc tags

Thank You!



KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)

Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A" Grade



JAVA PROGRAMMING

BY

Dr.T.Dhiliphan Rajkumar

Course Outline



CO 1. Understand the object oriented concepts using java



CO 2. Apply Object Oriented Concepts to develop Java Application



CO 3. Identify, Formulate and Analyze a Real World Problem and Develop a solution using Java Programming Concepts



CO 4. Ability to Work individually as well as in team to solve problem



CO 5. Communicate effectively with technical community during Interviews

Course description

Java still has its importance and popularity in the software industry, in spite of recent development of several high level languages. It has an excellent support of high-level as well as low-level functionality, which makes it suitable for many applications. This course provides students with a comprehensive study of the Java programming language. Instead of Classroom lectures, practical classes stress the strengths of Java, which provide the students with the means of writing efficient, maintainable, and portable code. A course with mini project is one of the great learning opportunities to develop students in various aspects. This course will also help the students during their placement sittings, as most of the companies test proficiency in programming using Java.

UNIT I OOP IN JAVA & INHERITANCE

Object Oriented Programming Concepts - OOP in Java - Characteristics of Java -Fundamental Programming Structures in Java - Defining classes in Java - Comments, Data Types, Variables, Operators, Control Flow, Arrays - constructors, methods -access specifiers - static members - Packages- Inheritance - Super classes- sub classes -Protected members - constructors in sub classes- Strings.

UNIT II EXCEPTION HANDLING AND I/O

Exceptions - exception hierarchy - throwing and catching exceptions - built-in exceptions, creating own exceptions, Stack Trace Elements-Input / Output Basics - Streams - Byte streams and Character streams - Reading and Writing Console - Reading and Writing Files - abstract classes and methods - final methods and classes

UNIT III INTERFACES AND MULTITHREADING

Interfaces - defining an interface, implementing interface, differences between classes and interfaces - extending interfaces - Object cloning - inner classes-Differences between multi-threading and multitasking, thread life cycle, creating threads, synchronizing threads, Inter-thread communication, daemon threads - Generic Programming.

UNIT IV AWT AND EVENT DRIVEN PROGRAMMING

AWT Event Hierarchy- Components - Graphics programming - Applets-Frame -working with 2D shapes - Using color, fonts, and images - Basics of event handling - event handlers - adapter classes - actions - mouse events - Introduction to Swing - layout management - Swing Components - Windows -Menus - Dialog Boxes.

UNIT V NETWORKING AND JDBC

Networking Basics - The Networking Classes and Interfaces -TCP/IP Client Sockets URL TCP/IP Server Sockets - **Datagrams** - A Relational Database Overview - JDBC Introduction - JDBC Product Components - JDBC Architecture - Case studies.



Unit II

Outcomes

- Handle exceptions
- Create own Exception handlers
- Get Input from command line
- Read/Write I/O using files
- Use Abstract and Final Methods and Classes

Unit 2 Outline



Lesson 1. Exceptions



Lesson 2. Built In and Custom Exceptions



Lesson 3. Byte Stream



Lesson 4. Character Stream



Lesson 5. Abstract and Final

Describes the exception handling mechanisms, different types of exceptions and their hierarchy, way to obtain command line arguments as inputs both numbers and characters, programming for files and abstract & final keywords.

Course Progress (use before starting new lesson)

Lesson 1. Exceptions



Lesson 2. Built in and Custom Exceptions



Lesson 3. Byte Stream



Lesson 4. Character Stream



Lesson 5. Abstract and Final

Motivation

- We seek robust programs
- When something unexpected occurs
 - Ensure program detects the problem
 - Then program must do something about it
- Extensive testing of special situations can result in "spaghetti code"
- Need mechanism to check for problem where it could occur
- When condition does occur
 - Have control passed to code to handle the problem

Exceptions

The exception handling in java is one of the powerful mechanism to handle the runtime errors so that normal flow of the application can be maintained.

An exception is an event, which occurs during the execution of a program that interrupts the normal flow of the program. It is an error thrown by a class or method reporting an error in code.

The process of converting system error messages into user friendly error message is known as **Exception handling**.

This is one of the powerful features of Java to handle run time error and maintain normal flow of java application.



Exception and Exception handling

Exception

Exception is an abnormal condition. In java, exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.

Exception handling

Exception Handling is a mechanism to handle runtime errors such as ClassNotFoundException, IO, SQL, Remote etc.

Advantage of Exception Handling

The core advantage of exception handling is to maintain the normal flow of the **application**. Exception normally disrupts the normal flow of the application that is why we use exception handling. Let's take a scenario:

statement 1;

statement 2; //exception occurs

statement 3;

statement 4;

statement 5;

Suppose there is 5 statements in your program and there occurs an exception at statement 2, rest of the code will not be executed i.e. statement 3 to 5 will not run. If we perform exception handling, rest of the statement will be executed. That is why we use exception handling in java.

When an exception can occur?

Exception can occur in two ways

- at runtime (known as runtime exceptions)
- at compile-time (known as Compile-time exceptions).

Reasons for Exceptions

There can be several reasons for an exception. For example, following situations can cause an exception

- Opening a non-existing file
- Network connection problem
- Operands being manipulated are out of prescribed ranges

Exception Handling

- When an exceptional condition arises, an object representing that exception is created and thrown in the method that caused the error.
- That method may choose to handle the exception itself, or pass it on. Either way, at some point, the exception is caught and processed.
- Java exception handling is managed via five keywords: **try**, **catch**, **throw**, **throws**, and **finally**.
- Program statements that you want to monitor for exceptions are contained within a **try block**. If an exception occurs within the try block, it is thrown.
- Your code can **catch** this exception (using catch) and handle it in some rational manner. System-generated exceptions are automatically thrown by the Java run-time system.

- To manually throw an exception, use the keyword **throw**. Any exception that is thrown out of a method must be specified as such by a **throws** clause.
- Any code that absolutely must be executed after a try block completes is put in a **finally** block.
- Code within try/catch block is referred as protected code. Syntax is as follows

```
try { // Protected code }  
catch (ExceptionName e1)  
    { // Catch block }  
Finally ( // block to be executed at end }
```

Throwing and Catching Exceptions

- The code which is prone to exceptions is placed in the try block.
- When an exception occurs, that exception occurred is handled by catch block associated with it.
- Every try block should be immediately followed either by a catch block or finally block.
- A catch statement involves declaring the type of exception you are trying to catch.
- If an exception occurs in protected code, the catch block (or blocks) that follows the try is checked.
- If the type of exception that occurred is listed in a catch block, the exception is passed to the catch block much as an argument is passed into a method parameter.

Single Catch

```
class Exc2 {  
    public static void main( String args []) { int d, a;  
    try { // monitor a block of code.  
        d = 0;  
        a = 42 / d;  
        System . out. println (" This will not be printed .");  
    }  
    catch ( ArithmeticException e) // divide - by- zero error  
    {  
        System . out. println (" Division by zero.");  
    }  
    System . out. println (" After catch statement.");  
    }  
}
```

Using Multiple Catch

- In some cases, more than one exception could be raised by a single piece of code. To handle this type of situation, you can specify two or more catch clauses, each catching a different type of exception.
- When an exception is thrown, each catch statement is inspected in order, and the first one whose type matches that of the exception is executed.
- After one catch statement executes, the others are bypassed, and execution continues after the try / catch block.

Example of Multiple Catch

```
class MultipleCatches
{
    public static void main( String args [])
    { try {
        int a = args. length ;

        System . out. println (" a  = " + a);  int b = 42 / a;

        int c[] = { 1 };  c[42] = 99;
    }
    catch ( ArithmeticException e)
    {
        System . out. println (" Divide by 0: " + e);
    }

    catch ( Array Index Out OfBounds Exception e)
    {

        System . out. println (" Array index oob: " + e);
    }
    System . out. println (" After try/ catch blocks.");
}
```

Usage of throw

- It is possible for your program to throw an exception explicitly, using the **throw** statement.
- The general form of throw is shown here: **throw ThrowableInstance;**
- Here, ThrowableInstance must be an object of type Throwable or a subclass of Throwable.
- Primitive types, such as int or char, as well as non-Throwable classes, such as String and Object, cannot be used as exceptions.
- The flow of execution stops immediately after the throw statement; any subsequent statements are not executed.
- The nearest enclosing try block is inspected to see if it has a catch statement that matches the type of exception.
- If it does find a match, control is transferred to that statement. If not, then the next enclosing try statement is inspected, and so on.
- If no matching catch is found, then the default exception handler halts the program and prints the stack trace.

Usage of throw - Example

```
class Throw Demo {  
    static void demoproc () { try {  
        throw new NullPointerException (" demo");  
    } catch ( NullPointerException e) { System . out. println (" Caught inside demoproc."); throw  
e; // rethrow the exception  
    }  
}  
    public static void main( String args []) { try {  
        demoproc ();  
    } catch ( NullPointerException e) { System . out. println (" Recaught: " + e);  
    }  
}
```

Usage of throws

- If a method is capable of causing an exception that it does not handle, it must specify this behavior so that callers of the method can guard themselves against that exception.
- You do this by including a throws clause in the method's declaration.
- A throws clause lists the types of exceptions that a method might throw.
- This is necessary for all exceptions, except those of type Error or RuntimeException, or any of their subclasses.
- All other exceptions that a method can throw must be declared in the throws clause.
- If they are not, a compile-time error will result.
- This is the general form of a method declaration that includes a throws clause:

```
type method-name(parameter-list) throws exception-list  
{  
// body of method  
}
```

- Here, exception-list is a comma-separated list of the exceptions that a method can throw.

Usage of throws - Example

```
class ThrowsDemo {  
    static void throw One () throws IllegalAccessException { System . out. println (" Inside throw  
One.");  
    throw new IllegalAccessException (" demo");  
}  
  
    public static void main( String args []) { try {  
        throw One ();  
    } catch ( IllegalAccessException e) { System . out. println (" Caught " + e);  
    }  
    }  
}
```

Usage of finally

- **finally** creates a block of code that will be executed after a try /catch block has completed and before the code following the try/catch block.
- The finally block will execute whether or not an exception is thrown.
- If an exception is thrown, the finally block will execute even if no catch statement matches the exception.

Example of finally



```
class Finally Demo { static void procA () {  
try {  
  
System . out . println (" inside procA "); throw new RuntimeException (" demo");  
} finally { System . out . println (" procA ' s finally ");  
}  
  
static void procB () { try {  
  
System . out . println (" inside procB "); return ;  
} finally { System . out . println (" procB ' s finally ");  
}  
  
public static void main( String args []) { try {  
procA ();  
} catch ( Exception e) { System . out . println (" Exception caught"); procB ();  
} }  
}
```

Topic 1
Exceptions

Topic 2
Exception Handling

Topic 3
Throwing and Catching Exceptions

Topic 4
Multiple Catch

Topic 5
Usage of throw, throws and finally

First Lesson Summary

Learned about Exception handling and their related terms

Course Progress (use before starting new lesson)



Lesson 1. Exceptions

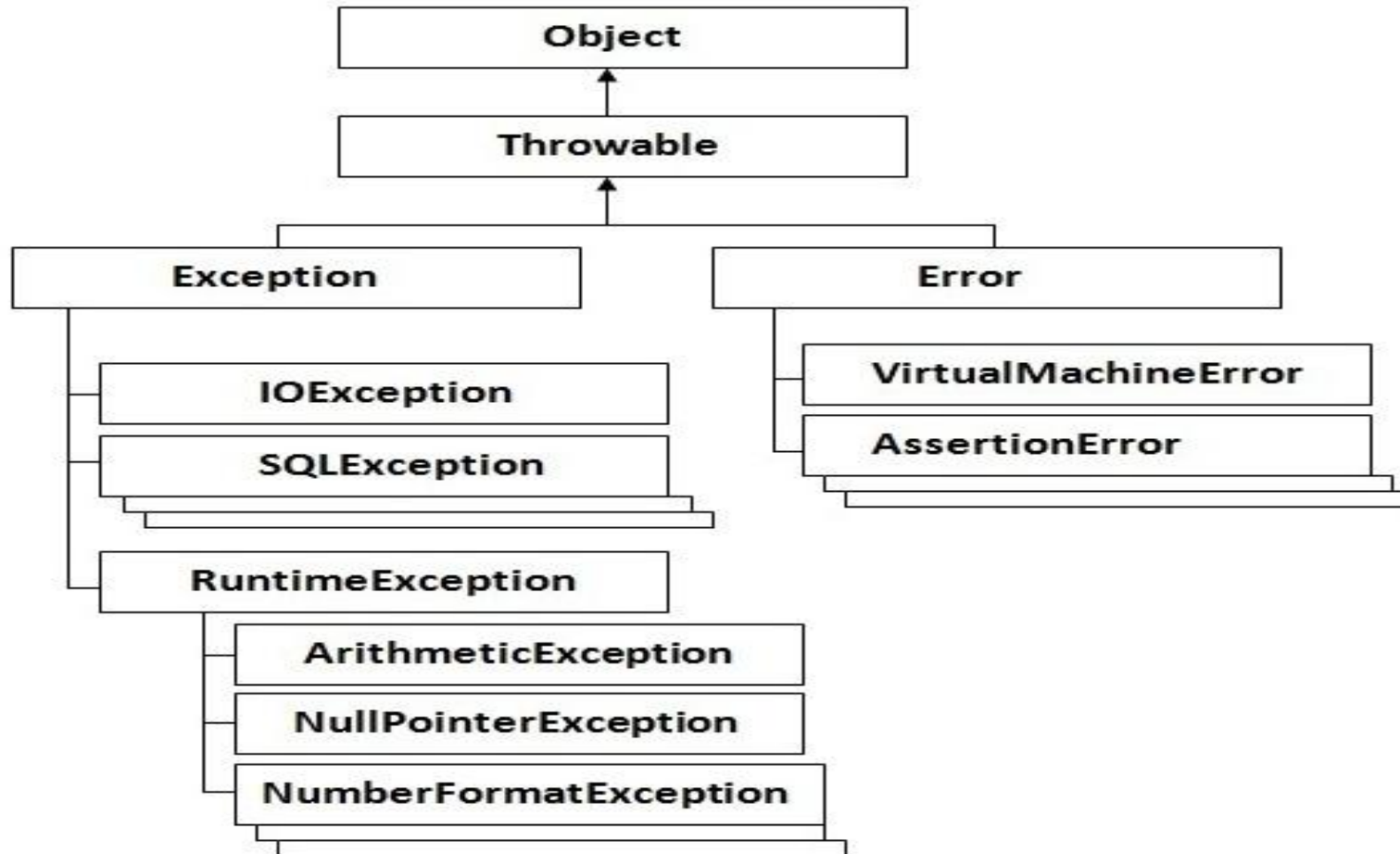
Lesson 2. Built in and Custom Exceptions

Lesson 3. Byte Stream

Lesson 4. Character Stream

Lesson 5. Abstract and Final

Hierarchy of Java Exception classes

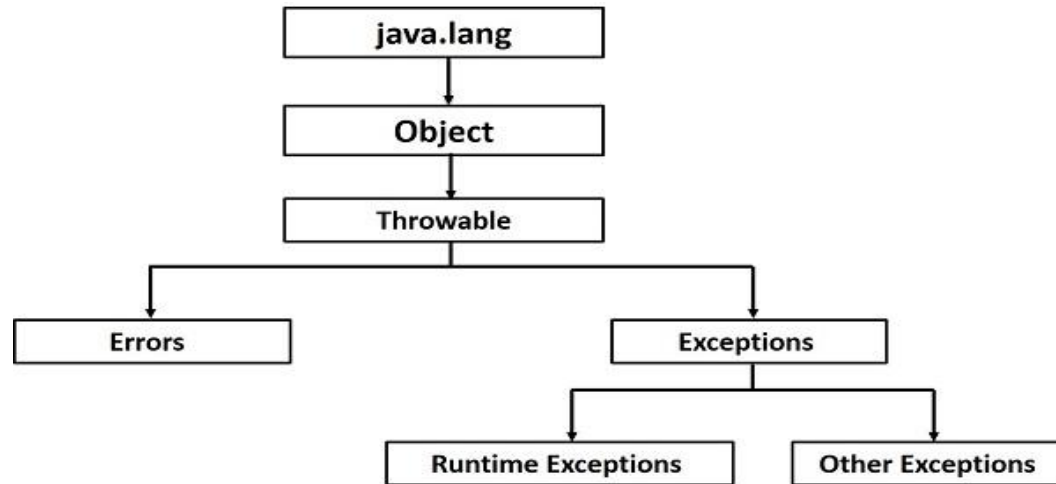


Types of Exceptions

There are mainly two types of exceptions: checked and unchecked where error is considered as unchecked exception. The sun microsystem says there are three types of exceptions:

- Checked Exception
- Unchecked Exception
- Error

Hierarchy of Java Exception



Checked, Unchecked and Error

1) Checked Exception

The classes that extend Throwable class except RuntimeException and Error are known as checked exceptions e.g. IOException, SQLException etc. Checked exceptions are checked at compile-time.

2) Unchecked Exception

The classes that extend RuntimeException are known as unchecked exceptions e.g. ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException etc. Unchecked exceptions are not checked at compile-time rather they are checked at runtime.

3) Error

Error is irrecoverable e.g. OutOfMemoryError, VirtualMachineError, AssertionError etc.

Common scenarios where exceptions may occur



1) Scenario where ArithmeticException occurs

If we divide any number by zero, there occurs an ArithmeticException.

```
int a=50/0;//ArithmeticException
```

2) Scenario where NullPointerException occurs

If we have null value in any variable, performing any operation by the variable occurs an NullPointerException.

```
String s=null;
```

```
System.out.println(s.length());//NullPointerException
```

Common scenarios Con"

3) Scenario where NumberFormatException occurs

The wrong formatting of any value, may occur NumberFormatException. Suppose I have a string variable that have characters, converting this variable into digit will occur NumberFormatException.

```
String s="abc";
```

```
int i=Integer.parseInt(s); //NumberFormatException
```

4) Scenario where ArrayIndexOutOfBoundsException occurs

If you are inserting any value in the wrong index, it would result ArrayIndexOutOfBoundsException as shown below:

```
int a[]=new int[5];
```

```
a[10]=50; //ArrayIndexOutOfBoundsException
```

Arithmetic exception

```
class ArithmeticException_Demo {  
    public static void main(String args[])  
    {  
        try {  
            int a = 30, b = 0;  
            int c = a / b; // cannot divide by zero  
            System.out.println("Result = " + c);  
        }  
        catch (ArithmeticException e) {  
            System.out.println("Can't divide a number by 0");  
        }  
    }  
}
```

Output

Can't divide a number by 0

ArrayIndexOutOfBoundsException Exception

```
class ArrayIndexOutOfBounds_Demo {  
    public static void main(String args[])  
    {  
        try {  
            int a[] = new int[5];  
            a[6] = 9; // accessing 7th element in an array of  
                // size 5  
        }  
        catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array Index is Out Of Bounds");  
        }  
    }  
}
```

Output

Array Index is Out Of Bounds

ClassNotFoundException

```
class B
{

}

class G
{

}

class MyClass {
public static void main(String[] args)
{
    Object o = class.forName(args[0]).newInstance();
    System.out.println("Class created for" + o.getClass().getName());
}
}
```

Output

ClassNotFoundException

Other standard Exceptions available

FileNotFoundException

IOException

InterruptedException

NoSuchMethodException

NullPointerException

NumberFormatException

StringIndexOutOfBoundsException

ClassCastException

StackOverflowError

NoClassDefFoundError

IllegalArgumentException

Exceptions Methods



Important Methods available in throwable class

- **public String getMessage()**

Returns a detailed message about the exception that has occurred. This message is initialized in the Throwable constructor.

- **public Throwable getCause()**

Returns the cause of the exception as represented by a Throwable object.

- **public String toString()**

Returns the name of the class concatenated with the result of getMessage().

Exceptions Methods Con”

- **public void printStackTrace()**

Prints the result of toString() along with the stack trace to System.err, the error output stream.

- **public StackTraceElement [] getStackTrace()**

Returns an array containing each element on the stack trace. The element at index 0 represents the top of the call stack, and the last element in the array represents the method at the bottom of the call stack.

- **public Throwable fillInStackTrace()**

Fills the stack trace of this Throwable object with the current stack trace, adding to any previous information in the stack trace.

Exception Hierarchy

All exception types are subclasses of the built-in class **Throwable**. Thus, **Throwable** is at the top of the exception class hierarchy. Immediately below **Throwable** are two subclasses that partition exceptions into two distinct branches.

One branch is headed by **Exception**.

- This class is used for exceptional conditions that user programs should catch.
- This is also the class that you will subclass to create your own custom exception types.
- There is an important subclass of **Exception**, called **RuntimeException**.
- Exceptions of this type are automatically defined for the programs that you write and include things such as division by zero and invalid array indexing.

Exception Hierarchy Con”



The other branch is topped by **Error**, which defines exceptions that are not expected to be caught under normal circumstances by your program.

- Exceptions of type **Error** are used by the Java run-time system to indicate errors having to do with the run-time environment, itself.
- Stack overflow is an example of such an error.

Built in Exceptions

Exception	Meaning
ArithmeticException	Arithmetic error, such as divide-by-zero.
ArrayIndexOutOfBoundsException	Array index is out-of-bounds.
ArrayStoreException	Assignment to an array element of an incompatible type.
ClassCastException	Invalid cast.
EnumConstantNotPresentException	An attempt is made to use an undefined enumeration value.
IllegalArgumentException	Illegal argument used to invoke a method.
IllegalMonitorStateException	Illegal monitor operation, such as waiting on an unlocked thread.
IllegalStateException	Environment or application is in incorrect state.
IllegalThreadStateException	Requested operation not compatible with current thread state.
IndexOutOfBoundsException	Some type of index is out-of-bounds.
NegativeArraySizeException	Array created with a negative size.
NullPointerException	Invalid use of a null reference.
NumberFormatException	Invalid conversion of a string to a numeric format.
SecurityException	Attempt to violate security.
StringIndexOutOfBoundsException	Attempt to index outside the bounds of a string.
TypeNotPresentException	Type not found.
UnsupportedOperationException	An unsupported operation was encountered.

Built in Exceptions

Exception	Meaning
ClassNotFoundException	Class not found.
CloneNotSupportedException	Attempt to clone an object that does not implement the Cloneable interface.
IllegalAccessException	Access to a class is denied.
InstantiationException	Attempt to create an object of an abstract class or interface.
InterruptedException	One thread has been interrupted by another thread.
NoSuchFieldException	A requested field does not exist.
NoSuchMethodException	A requested method does not exist.
ReflectiveOperationException	Superclass of reflection-related exceptions. (Added by JDK 7.)

Custom Exceptions

- To create your own exception types to handle situations specific to your applications: just define a subclass of Exception.
- Your subclasses don't need to actually implement anything—it is their existence in the type system that allows you to use them as exceptions.
- The Exception class does not define any methods of its own.
- It does, of course, inherit those methods provided by Throwable.
- Example while entering a if greater than 10 throws an exception

Custom Exceptions Example

```
class My Exception extends Exception {  
    private int detail;  
    My Exception ( int a) { detail = a; }  
    public String toString () {  
        return " My Exception [" + detail + "]; }  
    }  
class Exception Demo {  
    static void compute( int a) throws My Exception {  
        System . out. println (" Called compute(" + a + ")");  
        if( a > 10) throw new My Exception ( a);  
        System . out. println (" Normal exit");  
    }  
    public static void main( String args []) {  
        try { compute (1); compute (20);}  
        catch ( My Exception e) { System . out. println (" Caught " + e); }  
    }  
}
```

User-defined Custom Exception in Java

```
// A Class that represents use-defined expception
class MyException extends Exception
{
    public MyException(String s)
    {
        // Call constructor of parent Exception
        super(s);
    }
}

// A Class that uses above MyException
public class Main
{
    // Driver Program

    public static void main(String args[])
    {
        try
        {
            // Throw an object of user defined
            throw new MyException("Hello");
        }
        catch (MyException ex)
        {
            System.out.println("Caught");

            // Print the message from MyException
            System.out.println(ex.getMessage());
        }
    }
}
```

exception

object
Output
Caught
Hello

Topic 1
Types of Exceptions

Topic 2
Exception Methods

Topic 3
Exception Hierarchy

Topic 4
Built in Exceptions

Topic 5
Custom Exceptions

Second Lesson Summary

Understood and Use different types of built in Exceptions and custom exceptions

Course Progress (use before starting new lesson)



Lesson 1. Exceptions



Lesson 2. Built in and Custom Exceptions



Lesson 3. Byte Streams



Lesson 4. Character Streams

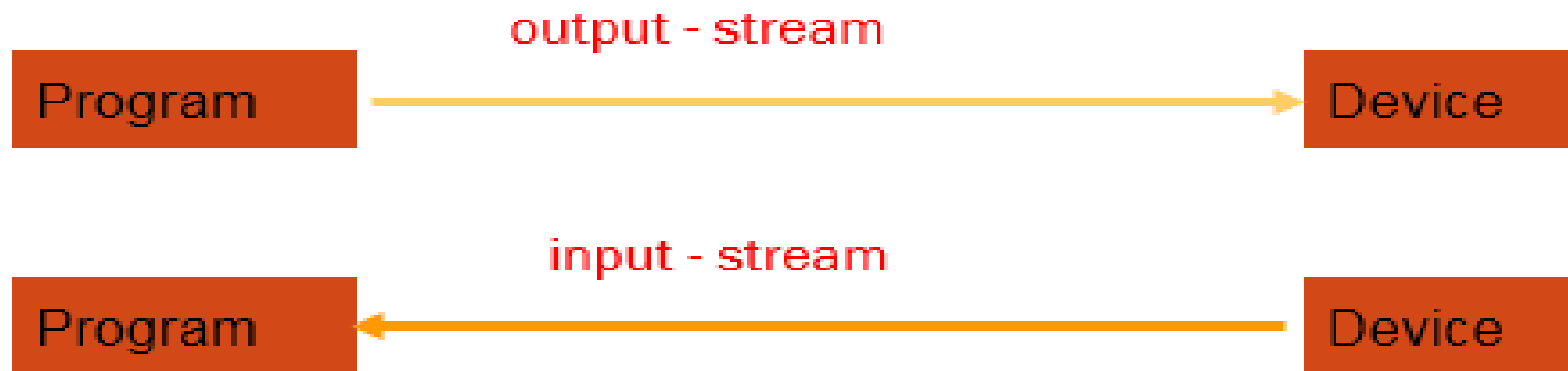


Lesson 5. Abstract and Final

I/O



- Usual Purpose: storing data to ‘nonvolatile’ devices, e.g. harddisk
- Classes provided by package java.io
- Data is transferred to devices by ‘streams’

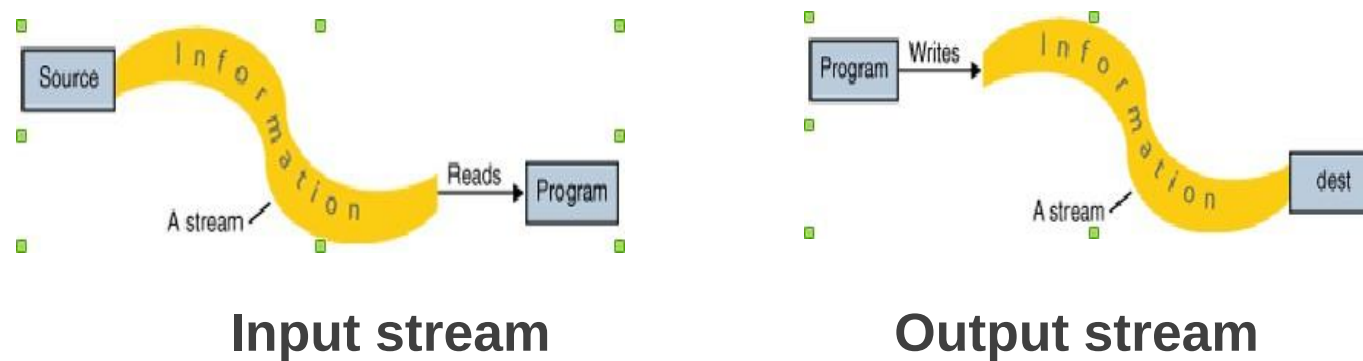


Stream

Java programs perform I/O through streams.

- **Stream:** is an abstraction that either produces or consumes information. A stream is linked to a physical device by the Java I/O system.
- **Input stream** can abstract many different kinds of input: **from a disk file, a keyboard, or a network socket.**
- **Output stream** may refer to the **console, a disk file, or a network connection.**

Java implements streams within class hierarchies defined in the **java.io** package.



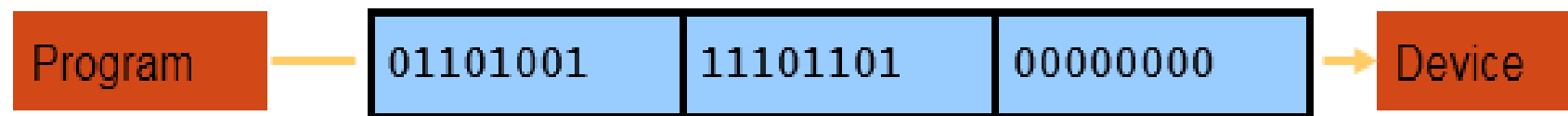
Overview of I/O streams

- The java.io package contains a collection of stream classes that support algorithms for reading and writing.
- To use these classes, a program needs to import the java.io package.
- The stream classes are divided into two class hierarchies, based on the data type (either characters or bytes) on which they operate
 - i.e Character Stream Classes
 - Byte Stream Classes

JAVA distinguishes between 2 types of streams:



- Binary Streams, containing 8 - bit information



I/O Streams Having



I/O Streams Having

- 2 types of streams (text / binary)
- 2 directions (input / output)

Results in 4 base-classes dealing with I/O:

1. InputStream: byte-input
2. OutputStream: byte-output
3. Reader: text-input
4. Writer: text-output

Streams

Java defines two types of streams: **byte** and **character**.

- **Byte streams:** provide a convenient means for handling input and output of bytes. Byte streams are used, for example, when reading or writing binary data.
- **Character streams:** provide a convenient means for handling input and output of characters. They use Unicode and, therefore, can be internationalized. Also, in some cases, character streams are more efficient than byte streams.
- **Byte streams** are defined by using two class hierarchies. **InputStream** and **OutputStream**.
- **Character streams** are defined by using two class hierarchies. **Reader** and **Writer**.

Byte Stream Classes

Stream Class	Meaning
BufferedInputStream	Buffered input stream
BufferedOutputStream	Buffered output stream
ByteArrayInputStream	Input stream that reads from a byte array
ByteArrayOutputStream	Output stream that writes to a byte array
DataInputStream	An input stream that contains methods for reading the Java standard data types
DataOutputStream	An output stream that contains methods for writing the Java standard data types
FileInputStream	Input stream that reads from a file
FileOutputStream	Output stream that writes to a file
FilterInputStream	Implements InputStream
FilterOutputStream	Implements OutputStream
InputStream	Abstract class that describes stream input
ObjectInputStream	Input stream for objects
ObjectOutputStream	Output stream for objects
OutputStream	Abstract class that describes stream output
PipedInputStream	Input pipe
PipedOutputStream	Output pipe
PrintStream	Output stream that contains print() and println()
PushbackInputStream	Input stream that supports one-byte “unget,” which returns a byte to the input stream
SequenceInputStream	Input stream that is a combination of two or more input streams that will be read sequentially, one after the other

InputStream vs OutputStream



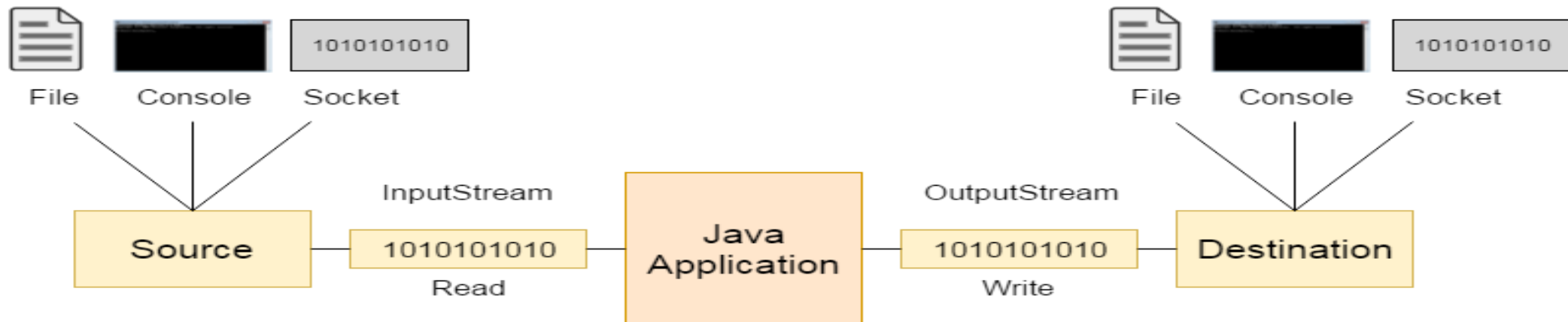
InputStream

Java application uses an input stream to read data from a source, it may be a file, an array, peripheral device or socket.

OutputStream

Java application uses an output stream to write data to a destination, it may be a file, an array, peripheral device or socket.

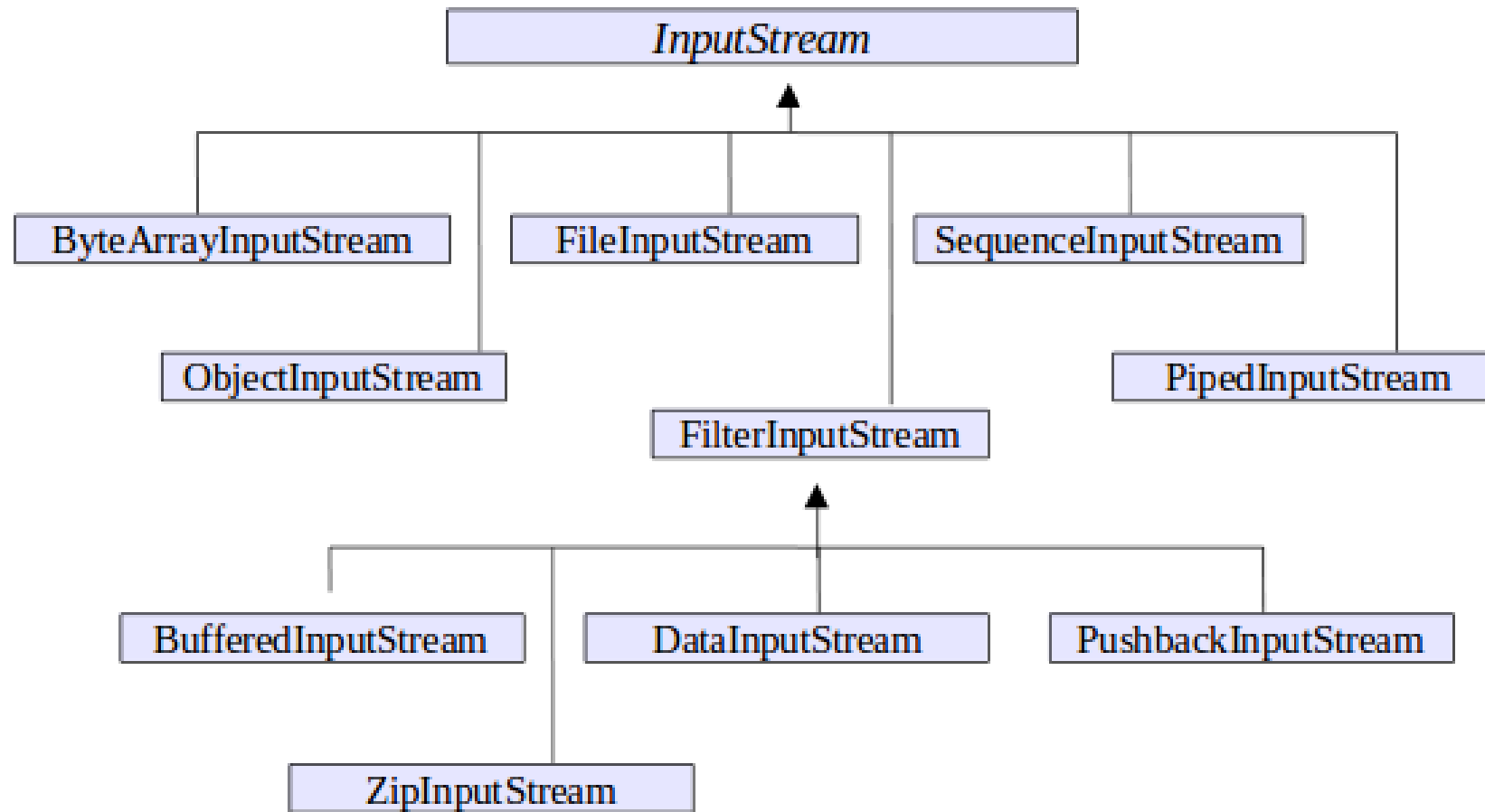
Let's understand working of Java OutputStream and InputStream by the figure given below.



an academy of research and education



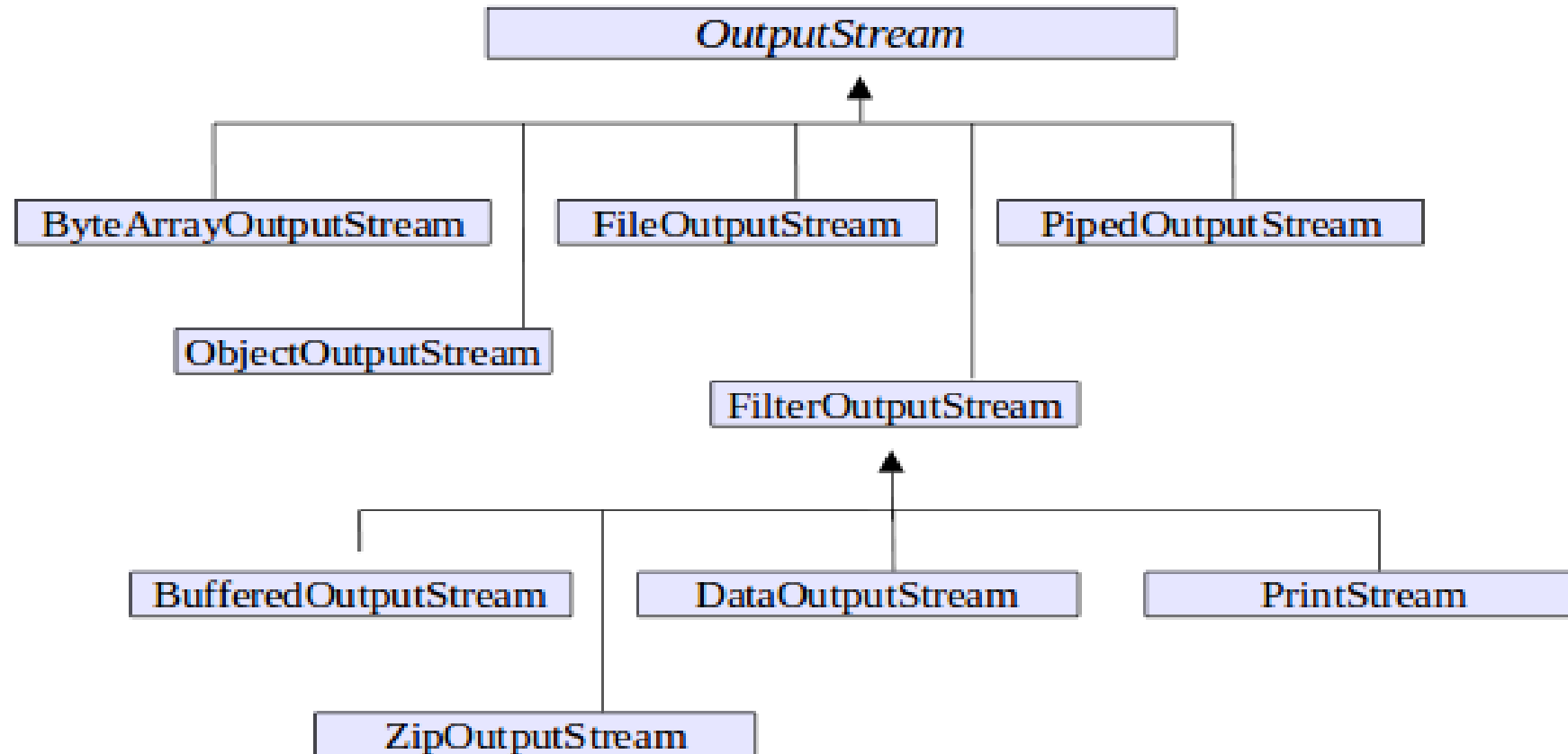
Byte Stream Classes-Input Stream



Common Method in InputStream

Method	Description
<code>public abstract <u>int</u> read() throws IOException</code>	reads the next byte of data from the input stream. It returns -1 at the end of file.
<code>public <u>int</u> available() throws IOException</code>	returns an estimate of the number of bytes that can be read from the current input stream.
<code>public void close() throws IOException</code>	is used to close the current input stream.

Byte Stream Classes-Output Stream



Common Method in OutputStream

Method	Description
1) <code>public void write(int) throws IOException</code>	is used to write a byte to the current output stream.
2) <code>public void write(byte[]) throws IOException</code>	is used to write an array of byte to the current output stream.
3) <code>public void flush() throws IOException</code>	flushes the current output stream.
4) <code>public void close() throws IOException</code>	is used to close the current output stream.

Methods in InputStream



- 1 int available() This method returns an estimate of the number of bytes that can be read (or skipped over) from this input stream without blocking by the next invocation of a method for this input stream.
- 2 void close() This method closes this input stream and releases any system resources associated with the stream.
- 3 void mark(int readlimit) This method marks the current position in this input stream.
- 4 boolean markSupported() This method tests if this input stream supports the mark and reset methods.
- 5 abstract int read() This method reads the next byte of data from the input stream.

Methods in InputStream Con"

6 int read(byte[] b) This method reads some number of bytes from the input stream and stores them into the buffer array b

7 int read(byte[] b, int off, int len) This method reads up to len bytes of data from the input stream into an array of bytes.

8 void reset() This method repositions this stream to the position at the time the mark method was last called on this input stream.

9 long skip(long n) This method skips over and discards n bytes of data from this input stream.

Methods in OutputStream

- 1 void close() This method closes this output stream and releases any system resources associated with this stream.
- 2 void flush() This method flushes this output stream and forces any buffered output bytes to be written out.
- 3 void write(byte[] b) This method writes *b.length* bytes from the specified byte array to this output stream.
- 4 void write(byte[] b, int off, int len) This method writes *len* bytes from the specified byte array starting at offset *off* to this output stream.
- 5 abstract void write(int b) This method writes the specified byte to this output stream.

BufferedInputStream Example

```
// Java program to demonstrate working of BufferedInputStream
import java.io.BufferedInputStream;
import java.io.FileInputStream;
import java.io.IOException;

// Java program to demonstrate BufferedInputStream methods
class BufferedInputStreamDemo
{
    public static void main(String args[]) throws IOException
    {
        // attach the file to FileInputStream
        FileInputStream fin = new FileInputStream("file1.txt");

        BufferedInputStream bin = new BufferedInputStream(fin);

        // illustrating available method
        System.out.println("Number of remaining bytes:" + bin.available());

        // illustrating markSupported() and mark() method
        boolean b=bin.markSupported();
        if (b)
            bin.mark(bin.available());
        // illustrating skip method

        /*Original File content:
        * This is my first line
        * This is my second line*/
        bin.skip(4);
```

```
// illustrating skip method
/*Original File content:
* This is my first line
* This is my second line*/
bin.skip(4);
System.out.println("FileContents :");

// read characters from FileInputStream and
// write them
int ch;
while ((ch=bin.read()) != -1)
    System.out.print((char)ch);

// illustrating reset() method
bin.reset();
while ((ch=bin.read()) != -1)
    System.out.print((char)ch);

// close the file
fin.close(); } }
```

Output: Number of remaining bytes:47

FileContents : is my first line This is my second line This is my first line This is my second line

BufferedOutputStream Example

```
//Java program demonstrating BufferedOutputStream

import java.io.*;

class BufferedOutputStreamDemo
{
    public static void main(String args[])throws Exception
    {
        FileOutputStream fout = new FileOutputStream("f1.txt");

        //creating bufferedOutputStream obj
        BufferedOutputStream bout = new BufferedOutputStream(fout);

        //illustrating write() method
        for(int i = 65; i < 75; i++)

        {
            bout.write(i);
        }

        byte b[] = { 75, 76, 77, 78, 79, 80 };
        bout.write(b);

        //illustrating flush() method
        bout.flush();

        //illustrating close() method
        bout.close();
        fout.close();
    } }
```

Output

ABCDEFGHIJKLMNOP

ByteArrayInputStream

The ByteArrayInputStream is composed of two words: ByteArray and InputStream.

As the name suggests, it can be used to read byte array as input stream.

Java ByteArrayInputStream class contains an internal buffer which is used to **read byte array** as stream.

In this stream, the data is read from a byte array.

The buffer of ByteArrayInputStream automatically grows according to data.

Java ByteArrayInputStream class constructors



Constructor	Description
<code>ByteArrayInputStream(byte[] ary)</code>	Creates a new byte array input stream which uses ary as its buffer array.
<code>ByteArrayInputStream(byte[] ary, int offset, int len)</code>	Creates a new byte array input stream which uses ary as its buffer array that can read up to specified len bytes of data from an array.

```
import java.io.*;

public class ReadExample
{
    public static void main(String[] args) throws IOException
    {
        byte[] buf = { 35, 36, 37, 38 };           // Create the new byte array input stream
        ByteArrayInputStream byt = new ByteArrayInputStream(buf);
        int k = 0;
        while ((k = byt.read()) != -1)
        {
            //Conversion of a byte into character
            char ch = (char) k;
            System.out.println("ASCII value of Character is:" + k + "; Special character is: " + ch);
        }
    }
}
```


Topic 1
Streams

Topic 2
Byte Streams

Topic 3
Buffered Stream

Third Lesson Summary

Use Byte streams for reading and writing

Course Progress (use before starting new lesson)



Lesson 1. Exceptions



Lesson 2. Built in and Custom Exceptions



Lesson 3. Byte Stream



Lesson 4. Character Stream

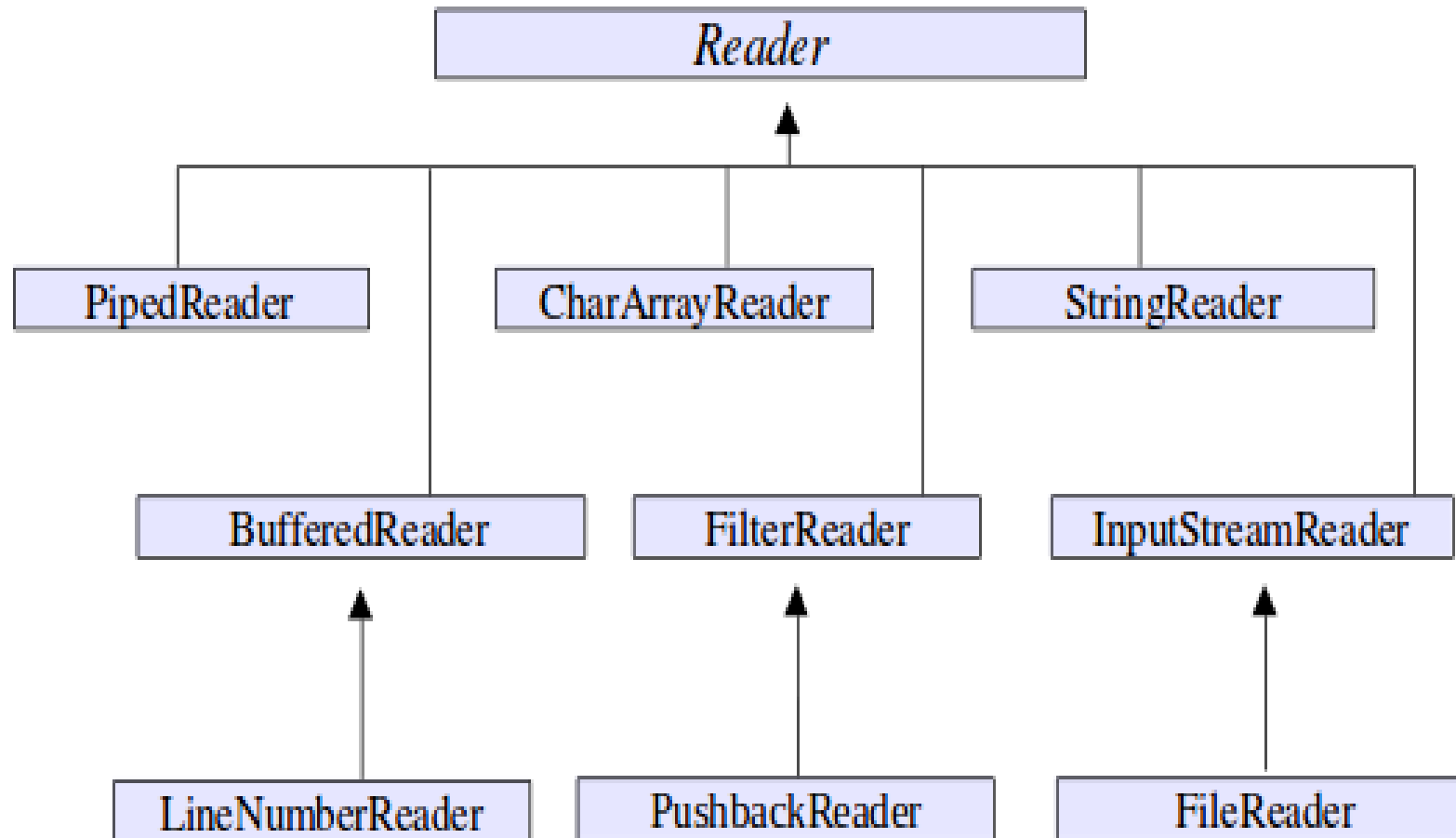


Lesson 5. Abstract and Final

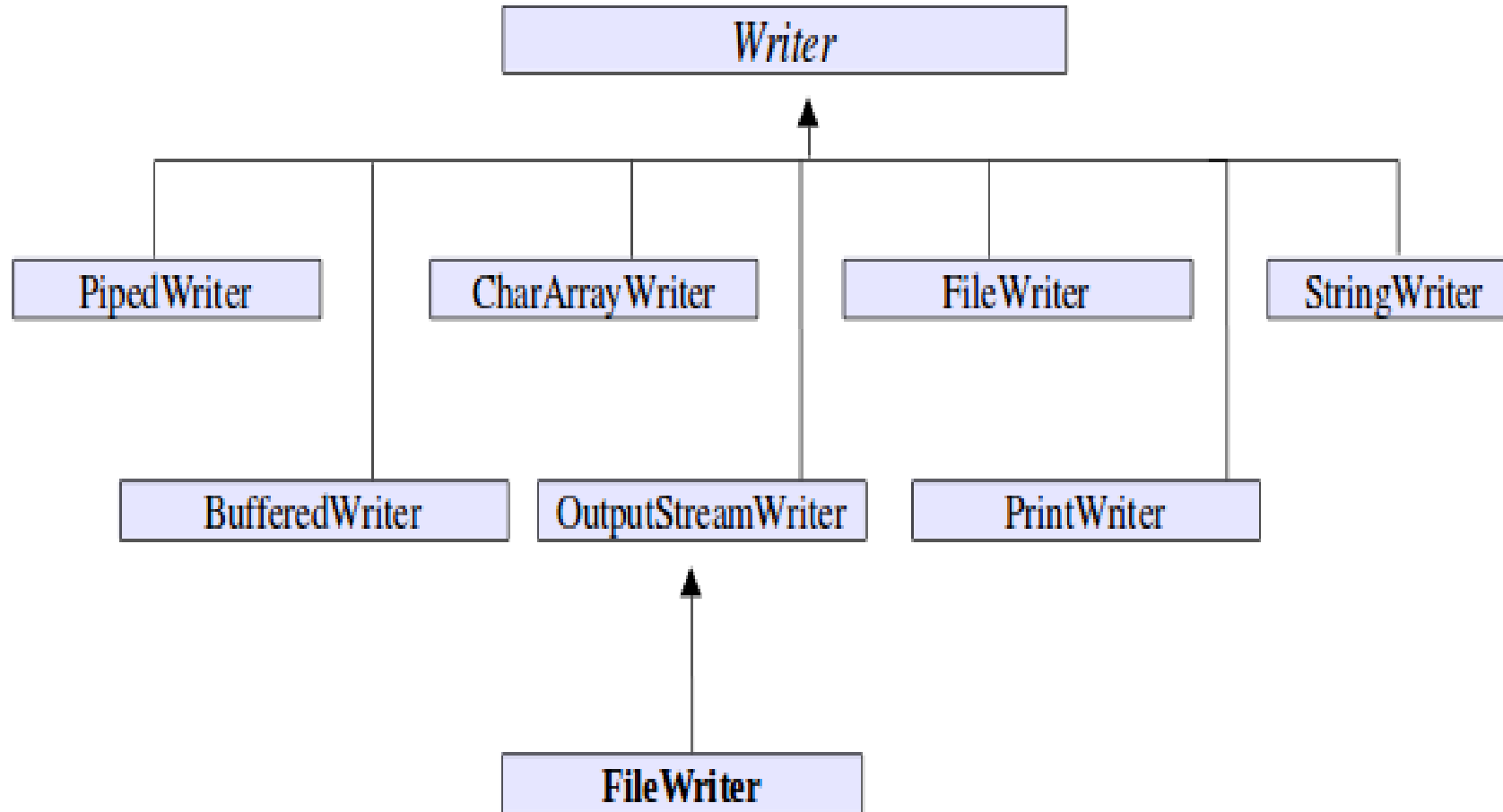
Character Streams

Stream Class	Meaning
BufferedReader	Buffered input character stream
BufferedWriter	Buffered output character stream
CharArrayReader	Input stream that reads from a character array
CharArrayWriter	Output stream that writes to a character array
FileReader	Input stream that reads from a file
FileWriter	Output stream that writes to a file
FilterReader	Filtered reader
FilterWriter	Filtered writer
InputStreamReader	Input stream that translates bytes to characters
LineNumberReader	Input stream that counts lines
OutputStreamWriter	Output stream that translates characters to bytes
PipedReader	Input pipe
PipedWriter	Output pipe
PrintWriter	Output stream that contains print() and println()
PushbackReader	Input stream that allows characters to be returned to the input stream
Reader	Abstract class that describes character stream input
StringReader	Input stream that reads from a string
StringWriter	Output stream that writes to a string
Writer	Abstract class that describes character stream output

Character Stream Classes: Reader



Character Stream Classes: Writer



BufferedInputStream

Java BufferedInputStream class is used to read information from stream.

It internally uses buffer mechanism to make the performance fast.

The important points about BufferedInputStream are:

- When the bytes from the stream are skipped or read, the internal buffer automatically refilled from the contained input stream, many bytes at a time.
- When a BufferedInputStream is created, an internal buffer array is created.

Methods



Method	Description
<code>int available()</code>	It returns an estimate number of bytes that can be read from the input stream without blocking by the next invocation method for the input stream.
<code>int read()</code>	It read the next byte of data from the input stream.
<code>int read(byte[] b, int off, int ln)</code>	It read the bytes from the specified byte-input stream into a specified byte array, starting with the given offset.
<code>void close()</code>	It closes the input stream and releases any of the system resources associated with the stream.
<code>void reset()</code>	It repositions the stream at a position the mark method was last called on this input stream.
<code>void mark(int readlimit)</code>	It sees the general contract of the mark method for the input stream.
<code>long skip(long x)</code>	It skips over and discards x bytes of data from the input stream.
<code>boolean markSupported()</code>	It tests for the input stream to support the mark and reset methods.

Java BufferedOutputStream Class



Java BufferedOutputStream class is used for buffering an output stream.

It internally uses buffer to store data.

It adds more efficiency than to write data directly into a stream. So, it makes the performance fast.

For adding the buffer in an OutputStream, use the BufferedOutputStream class.

Let's see the syntax for adding the buffer in an OutputStream:

```
OutputStream os= new BufferedOutputStream(new FileOutputStream("D:\\IO Package\\test  
out.txt"));
```


Reading Console Input



- Console input is accomplished by reading from **System.in**.
- To obtain a character-based stream that is attached to the console, wrap **System.in** in a **BufferedReader** object. **BufferedReader** supports a buffered input stream.
- A commonly used constructor is shown here:
BufferedReader(Reader inputReader)
- Here, **inputReader** is the stream that is linked to the instance of **BufferedReader** that is being created.
- **Reader** is an abstract class. One of its concrete subclasses is **InputStreamReader**, which converts bytes to characters.

Reading Console Input

- To obtain an `InputStreamReader` object that is linked to `System.in`, use the following constructor:

`InputStreamReader(InputStream inputStream)`

- Because `System.in` refers to an object of type `InputStream`, it can be used for `inputStream`.
- The following line of code creates a `BufferedReader` that is connected to the keyboard:

`BufferedReader br = new BufferedReader(new InputStreamReader(System.in));`

- After this statement executes, `br` is a character-based stream that is linked to the console through `System.in`.

Using BufferedReader to read characters from the console

```
import java. io .*;
class BRRead {
public static void main( String args []) throws IOException
{
char c;

Buffered Reader br = new Buffered Reader ( new InputStream Reader ( System . in));

System . out. println (" Enter characters , ' q' to quit.");
do {

c = ( char) br. read (); System . out. println ( c);
} while( c != ' q');
} }
```

Read a string from console using a BufferedReader

```
import java. io .*;
class BRReadLines {
public static void main( String args []) throws IOException
{
Buffered Reader br = new Buffered Reader ( new InputStream Reader ( System . in ));
String str;
System . out. println (" Enter lines of text.");
System . out. println (" Enter ' stop ' to quit.");
do {
str = br. readLine (); System . out. println ( str );
} while (! str. equals(" stop"));
} }
```

Read a number from console using a BufferedReader

```
import java. io .*;
public class UserInputInteger
{
public static void main( String args []) throws IOException
{
InputStream Reader read = new InputStream Reader ( System . in );
BufferedReader in = new BufferedReader ( read );
int number;

System . out. println (" Enter the number");
number = Integer. parseInt( in. readLine ());

System . out. println ( number );
}
}
```

Writing Console Output

```
class WriteDemo {  
    public static void main( String args [])  
    {  
        int b;  
        b = ' A';  
  
        System . out. write( b);  
  
        System . out. write('\ n');  
    }  
}
```

```
import java. io .*;  
public class PrintWriterDemo {  
    public static void main( String args []) {  
        PrintWriter pw = new PrintWriter ( System . out .true );  
        pw. println (" This is a string ");  
        int i = -7;  
        pw. println (i);  
        double d = 4.5 e -7;  
        pw. println ( d);  
    }  
}
```

Reading and Writing Files

- Two of the most often-used stream classes are **FileInputStream** and **FileOutputStream**, which create byte streams linked to files.
- To open a file, you simply create an object of one of these classes, specifying the name of the file as an argument to the constructor.
- Although both classes support additional constructors, the following are the forms that we will be using:

FileInputStream(String fileName) throws FileNotFoundException

FileOutputStream(String fileName) throws FileNotFoundException

Reading and Writing Files - Example

```
import java. io .*;
class CopyFile {

public static void main( String args []) throws IOException
{

int i; FileInputStream fin;

FileOutputStream fout;

try {
// open input file
try {
} fin = new FileInputStream ( args [0]);
catch ( FileNotFoundException e)
{ System . out. println (" Input File Not Found ");
return ; }
```


Reading and Writing Files – Example Contd.

```
try {  
    fout = new FileOutputStream ( args [1]);  
  
} catch ( FileNotFoundException e) { System . out. println (" Output File Error"); return ;  
}  
  
} catch ( Array Index Out OfBounds Exception e)  
{ System . out. println (" Usage: CopyFile From To");  
return ;}  
try {  
    do {  
        i = fin. read ();  
        if( i != -1) fout. write( i);  
        } while( i != -1);  
  
    } catch ( IOException e) { System . out. println (" File Error");  
    }  
    fin. close ();  
    fout. close ();  
}}
```

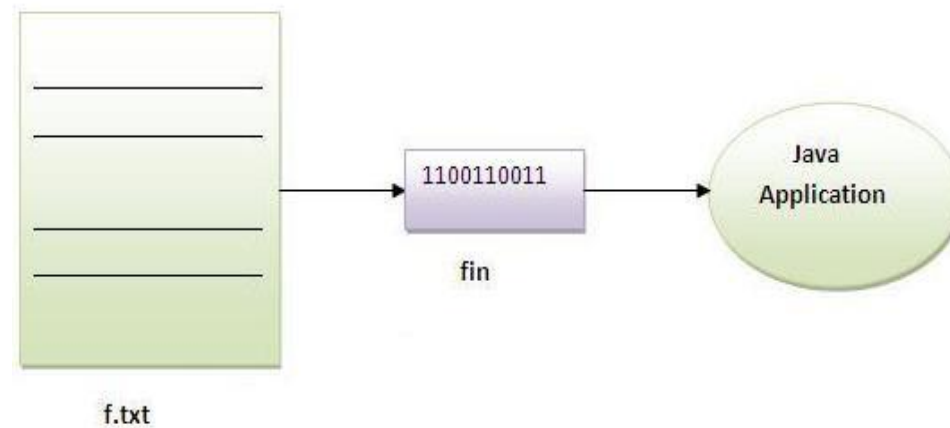
```
import java.io.FileInputStream;

public class DataStreamExample
{
    public static void main(String args[])
    {
        try
        {
            FileInputStream fin=new FileInputStream("D:\\testout.txt");

            int i=fin.read();

            System.out.print((char)i);

            fin.close();
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```



Note: Before running the code, a text file named as "**testout.txt**" is required to be created. In this file, we are having following content:

Hello welcomes you all for java programming.

After executing the above program, you will get a single character from the file which is 75 (in byte form).

To see the text, you need to convert it into character.

Output:

H

```
import java.io.*;
class SimpleRead
{
    public static void main(String args[])
    {
        try
        {
            FileInputStream fin=new FileInputStream("abc.txt");
            int i=0;
            while((i=fin.read())!=-1)
            {
                System.out.println((char)i);
            }
            fin.close();
        }
        catch(Exception e)    { system.out.println(e);    } } }
```

Output:
**Hello welcomes you all for java
programming.**

Topic 1
Character Streams

Topic 2
Reading & Writing Console I/O

Topic 3
BufferedReader

Topic 4
Reading and Writing Files

Fourth Lesson Summary

Use BufferedReader, Character and File Streams for reading and writing

Course Progress (use before starting new lesson)



Lesson 1. Exceptions



Lesson 2. Built in and Custom Exceptions



Lesson 3. Byte Stream



Lesson 4. Character Stream



Lesson 5. Abstract and Final

Abstract methods

- you can declare a *method* without defining it:

```
public abstract void draw(int size);
```

 - Notice that the body of the method is missing
- A method that has been declared but not defined is an abstract method

Abstract classes I

- Any class containing an abstract method is an abstract class
- You must declare the class with the keyword `abstract`:

```
abstract class MyClass {...}
```

- An abstract class is *incomplete*
 - It has “missing” method bodies
- You cannot instantiate (create a new instance of) an abstract class

Abstract classes II

- You can extend (subclass) an abstract class
 - If the subclass defines all the inherited abstract methods, it is “complete” and can be instantiated
 - If the subclass does *not* define all the inherited abstract methods, it too must be abstract
- You can declare a class to be abstract even if it does not contain any abstract methods
 - This prevents the class from being instantiated

Why have abstract classes?

- Suppose you wanted to create a class Shape, with subclasses Oval, Rectangle, Triangle, Hexagon, etc.
- You don't want to allow creation of a "Shape"
 - Only *particular* shapes make sense, not *generic* ones
 - If Shape is abstract, you can't create a new Shape
 - You *can* create a new Oval, a new Rectangle, etc.
- Abstract classes are good for defining a general category containing specific, "concrete" classes

An example abstract class

- public abstract class Animal
- {
 abstract int eat();
 abstract void breathe();
}
- This class cannot be instantiated
- Any non-abstract subclass of Animal must provide the eat() and breathe() methods

EXAMPLE

- `class Shape { ... }`
- `class Star extends Shape {
 void draw() { ... }
 ...
}`
- `class Crescent extends Shape {
 void draw() { ... }
 ...
}`
- `Shape someShape = new Star();`
 - This is legal, because a Star *is* a Shape
- `someShape.draw();`
 - This is a syntax error, because *some* Shape might not have a `draw()` method
 - Remember: ***A class knows its superclass, but not its subclasses***

Solution

- abstract class Shape {
 void draw();
}
- class Star extends Shape {
 void draw() { ... }
 ...
}
- class Crescent extends Shape {
 void draw() { ... }
 ...
}
- Shape someShape = new Star();
 - This is legal, because a Star *is* a Shape
 - However, Shape someShape = new Shape(); is *no longer* legal
- someShape.draw();
 - This is legal, because every actual instance *must* have a draw() method

Final Keyword



- The **final keyword** in java is used to restrict the user.
- The java final keyword can be used in many context.
- Final can be:
 - variable
 - method
 - class

- The final keyword can be applied with the variables, a final variable that have no value it is called blank final variable or uninitialized final variable.
- It can be initialized in the constructor only.
- The blank final variable can be static also which will be initialized in the static block only.
- Let's first learn the basics of final keyword.

Java final variable

- If you make any variable as final, you cannot change the value of final variable(It will be constant).
- **Example of final variable**
 - There is a final variable speed limit, we are going to change the value of this variable, but It can't be changed because final variable once assigned a value can never be changed.

EXAMPLE

```
class Bike9
{
    final int speedlimit=90; //final variable
    void run()
    {
        speedlimit=400;
    }
    public static void main(String args[])
    {
        Bike9 obj=new Bike9();
        obj.run();
    }
} //end of class
```

Java final method

- If you make any method as final, you cannot override it.

```
class Bike
```

```
{  
    final void run()  
    {  
        System.out.println("running");  
    }  
}
```

```
class Honda extends Bike
```

```
{  
    void run()  
    {System.out.println("running safely with 100kmph");}  
    public static void main(String args[])  
    {  
        Honda honda= new Honda();  
        honda.run();  
    }  
}
```

Java final class

- If you make any class as final, you cannot extend it.

```
final class Bike
```

```
{  
}
```

```
class Honda1 extends Bike
```

```
{
```

```
    void run()
```

```
{
```

```
        System.out.println("running safely with 100kmph");
```

```
}
```

```
    public static void main(String args[])
```

```
{
```

```
        Honda1 honda= new Honda();
```

```
        honda.run();
```

```
}
```

```
}
```

Is final method inherited?

- Yes, final method is inherited but you cannot override it.

```
class Bike
```

```
{
```

```
    final void run()
```

```
    {
```

```
        System.out.println("running...");
```

```
    }
```

```
}
```

```
class Honda2 extends Bike
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        new Honda2().run();
```

```
    }
```

```
}
```

What is blank or uninitialized final variable?



- A final variable that is not initialized at the time of declaration is known as blank final variable.
- If you want to create a variable that is initialized at the time of creating object and once initialized may not be changed, it is useful.
- For example PAN CARD number of an employee.
- It can be initialized only in constructor.

Can we initialize blank final variable?

Yes, but only in constructor.

```
class Bike10
{
    final int speedlimit; // blank final variable
    Bike10()
    {
        speedlimit=70;
        System.out.println(speedlimit);
    }
    public static void main(String args[])
    {
        new Bike10();
    }
}
```

static blank final variable

- A static final variable that is not initialized at the time of declaration is known as static blank final variable.
- It can be initialized only in static block.

```
class A
{
    static final int data; //static blank final variable
    static
    {
        data=50;
    }
    public static void main(String args[])
    {
        System.out.println(A.data);
    }
}
```

What is final parameter?

- If you declare any parameter as final, you cannot change the value of it.

```
class Bike11
{
    int cube(final int n)
    {
        n=n+2;//can't be changed as n is final
        n*n*n;
    }
    public static void main(String args[])
    {
        Bike11 b=new Bike11();
        b.cube(5);
    }
}
```


Can we declare a constructor final?

- No, because constructor is never inherited.

Topic 1
Abstract class

Topic 2
final keyword

Fifth Lesson Summary

Working of abstract and final keyword

Thank You!

OOP Concepts: Polymorphism, Inheritance, Abstraction

<https://gemini.google.com/app/cde824dd370a0605>

User prompt: Polymorphism: Method overloading and overriding – Dynamic method dispatch; Inheritance: Definition – Super and sub class, Types of inheritance: Single – Multilevel – Multiple – Hierarchical – Hybrid; Sub class constructors; Abstract classes – Interfaces – Defining and Implementing Interface - Collections -Object cloning. explain all these with codes

Response: This is a comprehensive overview of core Object-Oriented Programming (OOP) concepts, specifically using Java, as your terminology aligns perfectly with Java's architecture.

Here is a breakdown of each concept with clear explanations and code examples.

1. Polymorphism

Polymorphism means "many forms." It allows objects of different classes to be treated as objects of a common superclass, and it allows methods to do different things based on the object that it is acting upon.

- **Method Overloading (Compile-time Polymorphism):** Multiple methods in the same class share the same name but have different parameters (different type or number of arguments).
- **Method Overriding (Run-time Polymorphism):** A subclass provides a specific implementation for a method that is already defined in its superclass.
- **Dynamic Method Dispatch:** The mechanism where a call to an overridden method is resolved at *run-time* based on the actual object being referred to, rather than the reference type.

```
class MathOperations {
    // Method Overloading
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
}

class Animal {
    void makeSound() { System.out.println("Some generic animal sound"); }
}

class Dog extends Animal {
    // Method Overriding
    @Override
    void makeSound() { System.out.println("Bark!"); }
}

public class PolymorphismDemo {
    public static void main(String[] args) {
        // Dynamic Method Dispatch: Animal reference pointing to a Dog object
        Animal myAnimal = new Dog();
    }
}
```

```

        myAnimal.makeSound(); // Outputs: Bark! (Resolved at runtime)
    }
}

```

2. Inheritance

Inheritance is a mechanism where a new class (Subclass) acquires the properties and behaviors (fields and methods) of an existing class (Superclass).

- **Superclass:** The base/parent class being inherited from.
- **Subclass:** The derived/child class that inherits from the superclass.

Types of Inheritance

1. **Single:** One subclass inherits from one superclass ($A \rightarrow B$).
2. **Multilevel:** A subclass acts as a superclass for another class ($A \rightarrow B \rightarrow C$).
3. **Hierarchical:** Multiple subclasses inherit from a single superclass ($A \rightarrow B, A \rightarrow C$).
4. **Multiple:** One subclass inherits from multiple superclasses. *(Note: Java does not support this directly with classes to avoid ambiguity, but achieves it via Interfaces).*
5. **Hybrid:** A combination of two or more types of inheritance. *(Also handled via Interfaces in Java).*

```

// 1. Single Inheritance
class Vehicle {
    void drive() { System.out.println("Driving..."); }
}
class Car extends Vehicle { } // Car inherits Vehicle

// 2. Multilevel Inheritance
class SportsCar extends Car { } // SportsCar inherits Car, which inherits Vehicle

// 3. Hierarchical Inheritance
class Truck extends Vehicle { } // Both Car and Truck inherit Vehicle

```

3. Subclass Constructors

When a subclass object is created, the constructor of its superclass is called first. If you don't explicitly call it, the compiler automatically inserts `super()` to call the default (no-argument) superclass constructor. You can use `super(args)` to call a specific parameterized superclass constructor.

```

class Parent {
    Parent(String name) {
        System.out.println("Parent constructor for " + name);
    }
}

```

```

    }

    class Child extends Parent {
        Child() {
            // Explicitly calling the superclass constructor
            super("Alice");
            System.out.println("Child constructor");
        }
    }
}

```

4. Abstract Classes & Interfaces

Both are used to achieve abstraction—hiding the implementation details and showing only functionality to the user.

- **Abstract Class:** Declared with the `abstract` keyword. It can have both abstract methods (methods without a body) and concrete methods (methods with a body). You *cannot* create an object of an abstract class.
- **Interface:** A blueprint of a class. Traditionally, it only contains abstract methods and static constants. A class *implements* an interface to provide the behavior. This is how Java handles multiple inheritance.

```

// Abstract Class
abstract class Shape {
    abstract void draw(); // Abstract method (no body)

    void display() { // Concrete method
        System.out.println("Displaying shape...");
    }
}

// Interface (Defining)
interface Flyable {
    void fly(); // implicitly public and abstract
}

interface Swimmable {
    void swim();
}

// Implementing multiple interfaces (Multiple Inheritance)
class Duck implements Flyable, Swimmable {
    public void fly() { System.out.println("Duck is flying"); }
    public void swim() { System.out.println("Duck is swimming"); }
}

class Circle extends Shape {
    void draw() { System.out.println("Drawing a circle"); }
}

```

5. Collections

The Java Collections Framework provides an architecture to store and manipulate a group of objects. It includes interfaces (like List, Set, Map) and concrete implementation classes (like ArrayList, HashSet, HashMap).

```
import java.util.ArrayList;
import java.util.List;

public class CollectionsDemo {
    public static void main(String[] args) {
        // Using a List Collection to store Strings
        List<String> names = new ArrayList<>();
        names.add("Alice");
        names.add("Bob");

        for(String name : names) {
            System.out.println(name);
        }
    }
}
```

6. Object Cloning

Object cloning is the process of creating an exact copy of an object. In Java, this is done by implementing the Cloneable interface and overriding the clone() method from the Object class.

Note: By default, clone() performs a *shallow copy* (copies primitive values, but keeps references to the same objects for non-primitives).

```
// Must implement Cloneable to allow cloning
class Employee implements Cloneable {
    int id;
    String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    // Overriding clone method
    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}

public class CloneDemo {
    public static void main(String[] args) {

```

```

    try {
        Employee emp1 = new Employee(101, "John");

        // Cloning emp1 into emp2
        Employee emp2 = (Employee) emp1.clone();

        System.out.println(emp2.name); // Outputs: John
    } catch (CloneNotSupportedException e) {
        e.printStackTrace();
    }
}
}

```

User prompt: Object Oriented Programming Concepts - Object, Class, Abstraction, Encapsulation, Polymorphism – JVM -Characteristics of Java – Defining Classes and objects in Java – Access Specifiers, Comments, Data Types, Variables, Operators, Control Flow, Arrays – Constructors and types, Methods - Static members (variables and methods) – Packages- defining, creating, Importing packages- Scanner class; Type Casting. same explanation for these topics

Response: Here is a structured breakdown of the foundational Object-Oriented Programming (OOP) concepts and Java basics, complete with explanations and code examples.

1. Core OOP Concepts

- **Class:** A blueprint or template that defines the structure, data, and behavior that objects of a certain type will have.
- **Object:** An instance of a class. It represents a real-world entity with state (attributes) and behavior (methods).
- **Encapsulation:** The bundling of data (variables) and methods that operate on the data into a single unit (a class). It restricts direct access to some of the object's components, usually by making fields `private` and exposing `public` getter/setter methods.
- **Abstraction:** Hiding complex implementation details and showing only the essential features of an object. (Achieved via abstract classes and interfaces).
- **Polymorphism:** As covered previously, the ability of a method or object to take on many forms (Overloading and Overriding).

```

// Encapsulation, Class, and Object Demo
class BankAccount {
    // Encapsulated data (private)
    private double balance;

    // Constructor to initialize the object
    public BankAccount(double initialBalance) {
        this.balance = initialBalance;
    }
}

```



```
// Public method to access the private data (Abstraction/Encapsulation)
public void deposit(double amount) {
    if (amount > 0) balance += amount;
}

public double getBalance() {
    return balance;
}

}

public class OOPDemo {
    public static void main(String[] args) {
        // Creating an Object of the BankAccount class
        BankAccount myAccount = new BankAccount(1000.0);
        myAccount.deposit(500.0);
        System.out.println("Balance: $" + myAccount.getBalance());
    }
}
```

2. JVM and Characteristics of Java

Java Virtual Machine (JVM): The engine that provides a runtime environment to drive the Java Code or applications. It converts Java bytecode (compiled code) into machine-specific language. This is what makes Java platform-independent.

Characteristics of Java:

- **Object-Oriented:** Everything is modeled as an object.
- **Platform-Independent:** "Write Once, Run Anywhere" (WORA). Compiled bytecode runs on any OS with a JVM.
- **Simple & Secure:** No explicit pointers, automatic memory management (Garbage Collection), and a secure runtime environment.
- **Robust:** Strong memory management and exception handling.

3. Java Building Blocks: Syntax & Rules

Access Specifiers

Determine the visibility of classes, methods, and variables.

1. **public:** Accessible from anywhere.
2. **private:** Accessible only within the same class.
3. **protected:** Accessible within the same package and subclasses.
4. **Default (No keyword):** Accessible only within the same package.

Comments, Variables, Data Types, and Operators

- **Comments:** Notes for developers ignored by the compiler. Single-line (`//`), Multi-line (`/* */`), JavaDoc (`/** */`).
- **Data Types:** Primitive (e.g., `int`, `double`, `boolean`, `char`) and Reference (e.g., `Objects`, `Arrays`, `Strings`).
- **Variables:** Containers for storing data values.
- **Operators:** Perform operations on variables (Arithmetic `+` `-` `*` `/`, Relational `==` `!=` `<` `>`, Logical `&&` `||` `!`).

Control Flow and Arrays

- **Control Flow:** Dictates the execution order (if-else, switch, for, while).
- **Arrays:** A collection of variables of the same type stored in contiguous memory.

```
public class BasicsDemo {
    public static void main(String[] args) {
        // Comments & Data Types
        int age = 25;           // Integer
        double price = 19.99;   // Decimal
        boolean isStudent = true; // Boolean
        char grade = 'A';       // Single character

        // Operators & Control Flow
        if (age >= 18 && isStudent) {
            System.out.println("Eligible for student discount.");
        } else {
            System.out.println("Standard pricing.");
        }

        // Arrays & Loops (Control Flow)
        int[] scores = {90, 85, 92, 78}; // Array initialization

        System.out.print("Scores: ");
        for (int i = 0; i < scores.length; i++) {
            System.out.print(scores[i] + " ");
        }
    }
}
```

4. Constructors, Methods, and Static Members

- **Constructors:** Special methods used to initialize objects. They share the exact name of the class and have no return type.
 - *Default Constructor:* Takes no arguments.

- *Parameterized Constructor*: Takes arguments to set initial values.
- **Methods**: Blocks of code that perform a specific task and only run when called.
- **Static Members**: Belong to the *class itself* rather than any specific object instance. A static variable is shared among all objects of that class.

```
class Student {
    String name;          // Instance variable
    static String schoolName = "Global High"; // Static variable (shared)

    // Default Constructor
    Student() {
        this.name = "Unknown";
    }

    // Parameterized Constructor
    Student(String name) {
        this.name = name;
    }

    // Instance Method
    void introduce() {
        System.out.println("Hi, I'm " + name + " from " + schoolName);
    }

    // Static Method
    static void changeSchool(String newName) {
        schoolName = newName; // Can only access static variables directly
    }
}
```

5. Packages, Scanner, and Type Casting

- **Packages**: A way to group related classes to avoid name conflicts and organize code. (e.g., `java.util`). You define them at the top of a file with `package com.myproject`; and import them with `import java.util.Scanner`;
- **Scanner Class**: A utility class in `java.util` used for getting user input from the console.
- **Type Casting**: Converting one primitive data type into another.
 - *Implicit (Widening)*: Automatic conversion of a smaller type to a larger type size (e.g., `int` to `double`).
 - *Explicit (Narrowing)*: Manual conversion of a larger type to a smaller size using parentheses (e.g., `double` to `int`).

```
// Importing a package
import java.util.Scanner;
```

```
public class InputAndCasting {  
    public static void main(String[] args) {  
        // Type Casting  
        int myInt = 9;  
        double myDouble = myInt;          // Implicit Casting (Widening): 9.0  
  
        double pi = 3.14159;  
        int truncatedPi = (int) pi;       // Explicit Casting (Narrowing): 3  
  
        // Scanner Class for User Input  
        Scanner scanner = new Scanner(System.in);  
  
        System.out.print("Enter your username: ");  
        String userName = scanner.nextLine(); // Reads a string  
  
        System.out.println("Welcome, " + userName + "!");  
  
        // Always close the scanner when done  
        scanner.close();  
    }  
}
```
